

ML4NLP_2

January 13, 2025

1 Data Cleaning

1.0.1 Load file from Drive

```
[10]: from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
[38]: import os
import zipfile

project_dir = '/content/drive/MyDrive/Colab Notebooks/A5 - ESILV/Machine_
↳ Learning for NLP/Project 2'
zip_path = os.path.join(project_dir, 'Traduction avis clients.zip')
extract_path = os.path.join(project_dir, 'extracted', 'Traduction avis clients')
```

```
[39]: # Extract the ZIP file
# with zipfile.ZipFile(zip_path, 'r') as zip_ref:
#     zip_ref.extractall(extract_path)
```

```
[40]: import pandas as pd
from collections import Counter
from wordcloud import WordCloud
import matplotlib.pyplot as plt
from textblob import TextBlob

# glob enables to load all files from a folder
import glob

files = glob.glob(os.path.join(extract_path, "*.xlsx"))
all_reviews = []

for file in files:
    df = pd.read_excel(file)
    all_reviews.append(df)
```

```
[41]: # Concatenate all reviews into a single dataframe
full_data = pd.concat(all_reviews, ignore_index=True)
print("Loaded data shape:", full_data.shape)
```

Loaded data shape: (34435, 11)

```
[42]: full_data.head()
```

```
[42]:
```

	note	auteur \		avis	assureur \				
0	1.0	maitre-en-colere-114722							
1	5.0	fredo-102557							
2	4.0	louisonne-f-130215							
3	3.0	stephane-p-128761							
4	1.0	montad-137724							

0	Pour tous les maîtres des animaux\n L'assuranc...			Eca Assurances					
1	25 ans chez AMV pour ma moto et mes voitures, ...			AMV					
2	Je suis satisfait de votre prestation et servi...	L'olivier Assurance							
3	SATISFAIT ET FACILE DE SOUSCRIRE UN CONTRAT EN...			AMV					
4	Je croyais qu'allianz était une bonne assuranc...			Allianz					

	produit	type	date_publication	date_exp \
0	animaux	train	25/05/2021	01/05/2021
1	moto	train	13/01/2021	01/01/2021
2	auto	train	30/08/2021	01/08/2021
3	moto	train	20/08/2021	01/08/2021
4	auto	train	18/10/2021	01/10/2021

		avis_en	avis_cor	avis_cor_en
0	For all animal owners\n ECA insurance And avoi...	NaN		NaN
1	25 years at AMV for my motorcycle and my cars,...	NaN		NaN
2	I am satisfied with your performance and servi...	NaN		NaN
3	SATISFIED AND EASY TO SUBSCRIBE TO AN ONLINE C...	NaN		NaN
4	I thought Allianz was good insurance but I was...	NaN		NaN

```
[43]: full_data.to_csv(os.path.join(project_dir, 'full_data.csv'), index=False)
```

```
[15]: full_data.describe()
```

```
[15]:
```

	note
count	24104.000000
mean	2.847619
std	1.531390
min	1.000000
25%	1.000000
50%	3.000000
75%	4.000000

max 5.000000

1.0.2 Distribution of the data for different variables

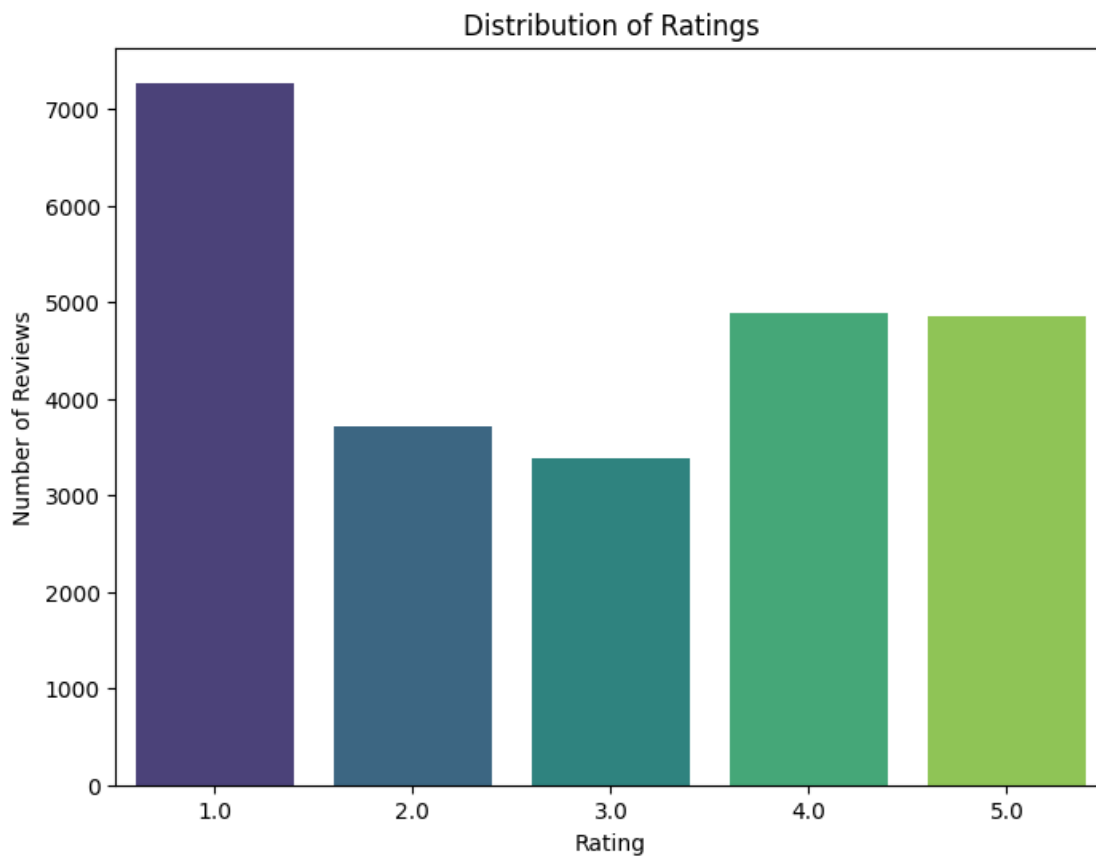
```
[16]: import seaborn as sns
```

```
[17]: plt.figure(figsize=(8, 6))
sns.countplot(x='note', data=full_data, palette='viridis')
plt.title('Distribution of Ratings')
plt.xlabel('Rating')
plt.ylabel('Number of Reviews')
plt.show()
```

<ipython-input-17-c62c354fd4b7>:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(x='note', data=full_data, palette='viridis')
```

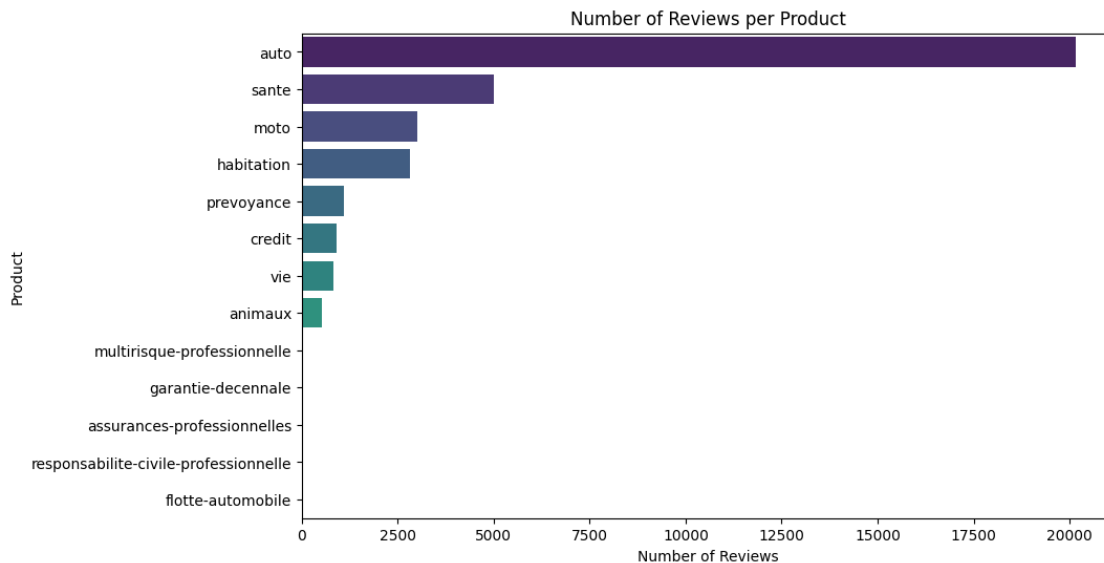


```
[18]: plt.figure(figsize=(10, 6))
sns.countplot(y='produit', data=full_data, order=full_data['produit'].
↪value_counts().index, palette='viridis')
plt.title('Number of Reviews per Product')
plt.xlabel('Number of Reviews')
plt.ylabel('Product')
plt.show()
```

<ipython-input-18-8a247f2fe7c5>:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(y='produit', data=full_data,
order=full_data['produit'].value_counts().index, palette='viridis')
```



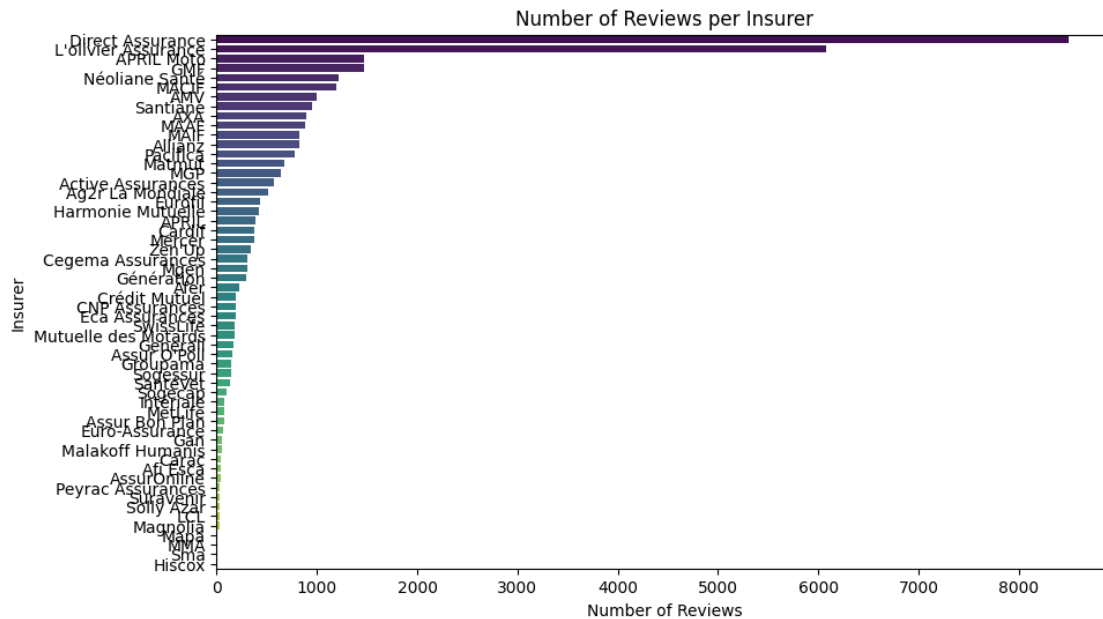
```
[19]: plt.figure(figsize=(10, 6))
sns.countplot(y='assureur', data=full_data, order=full_data['assureur'].
↪value_counts().index, palette='viridis')
plt.title('Number of Reviews per Insurer')
plt.xlabel('Number of Reviews')
plt.ylabel('Insurer')
plt.show()
```

<ipython-input-19-43ecc2976d1e>:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same

effect.

```
sns.countplot(y='assureur', data=full_data,
order=full_data['assureur'].value_counts().index, palette='viridis')
```



```
[20]: # Convert 'date_publication' to datetime
full_data['date_publication'] = pd.to_datetime(full_data['date_publication'],
dayfirst=True)

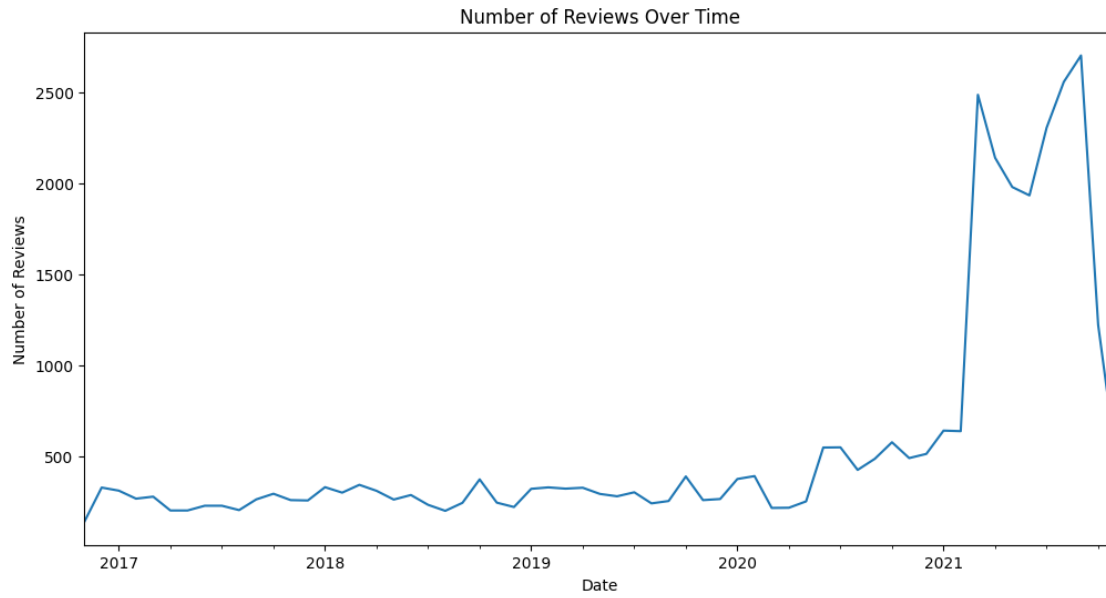
# Set date as index
full_data.set_index('date_publication', inplace=True)

monthly_reviews = full_data['avis'].resample('M').count() # for monthly count

plt.figure(figsize=(12, 6))
monthly_reviews.plot()
plt.title('Number of Reviews Over Time')
plt.xlabel('Date')
plt.ylabel('Number of Reviews')
plt.show()
```

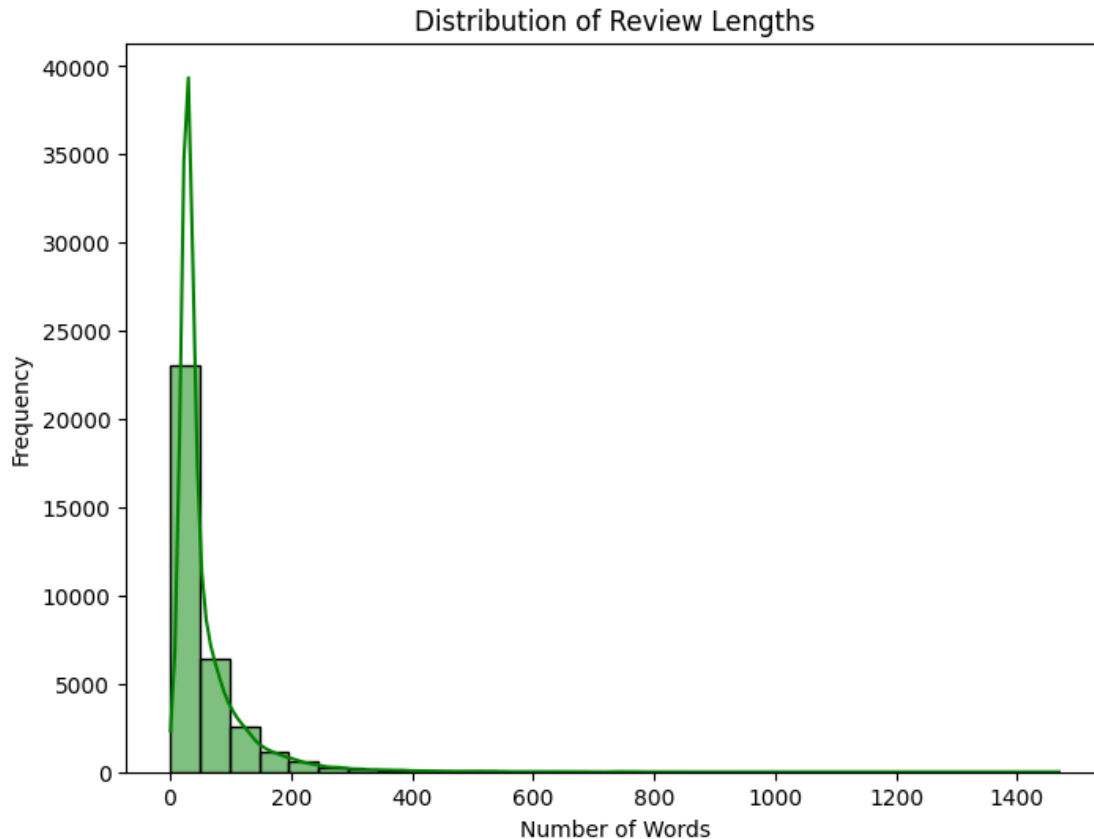
<ipython-input-20-612eefb48d2c>:7: FutureWarning: 'M' is deprecated and will be removed in a future version, please use 'ME' instead.

```
monthly_reviews = full_data['avis'].resample('ME').count() # for monthly count
```



```
[21]: # Number of words in each review
full_data['avis_length'] = full_data['avis'].apply(lambda x: len(str(x).
↪split()))

plt.figure(figsize=(8, 6))
sns.histplot(full_data['avis_length'], bins=30, kde=True, color='green')
plt.title('Distribution of Review Lengths')
plt.xlabel('Number of Words')
plt.ylabel('Frequency')
plt.show()
```



```
[25]: from wordcloud import WordCloud
      from nltk.corpus import stopwords
      import nltk
      nltk.download('stopwords')
```

```
      from spacy.lang.fr.stop_words import STOP_WORDS as fr_stop
      from spacy.lang.en.stop_words import STOP_WORDS as en_stop
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
```

```
[23]: # Combine all reviews into a single string
      text = " ".join(review for review in full_data['avis'].dropna())

      # Define french stopwords
      stopwords = fr_stop

      wordcloud = WordCloud(stopwords=stopwords, background_color='white', width=800,
      ↪ height=400).generate(text)
```

[illegible]

8



```
[27]: from sklearn.feature_extraction.text import CountVectorizer

def plot_ngrams(text_series, n=2, top=20):
    vec = CountVectorizer(ngram_range=(n, n), stop_words=list(fr_stop)).
    fit(text_series)
    bag_of_words = vec.transform(text_series)
    sum_words = bag_of_words.sum(axis=0)
    words_freq = [(word, sum_words[0, idx]) for word, idx in vec.vocabulary_.
    items()]
    words_freq = sorted(words_freq, key=lambda x: x[1], reverse=True)
    top_words = words_freq[:top]
    words, counts = zip(*top_words)
    plt.figure(figsize=(10, 6))
    sns.barplot(x=list(counts), y=list(words), palette='viridis')
    plt.title(f'Top {top} {n}-grams')
    plt.xlabel('Frequency')
    plt.ylabel(f'{n}-gram')
    plt.show()

# Top 20 bigrams
plot_ngrams(full_data['avis'].dropna(), n=2, top=20)
```

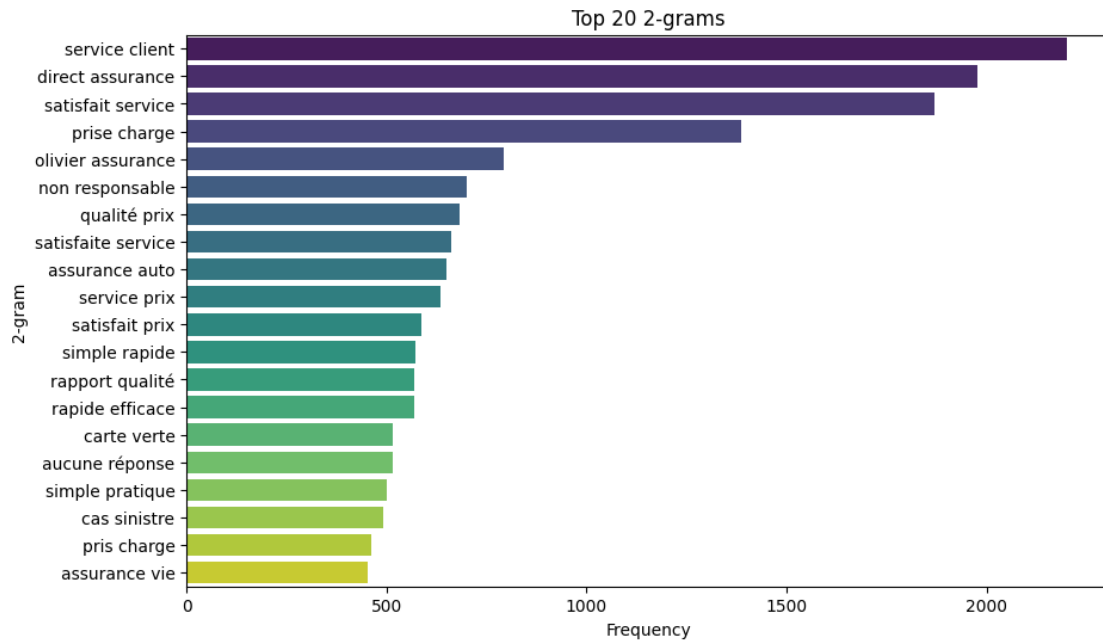
```

/usr/local/lib/python3.10/dist-packages/sklearn/feature_extraction/text.py:402:
UserWarning: Your stop_words may be inconsistent with your preprocessing.
Tokenizing the stop words generated tokens ['neuf', 'qu', 'quelqu'] not in
stop_words.
  warnings.warn(
<ipython-input-27-1f11099f67e0>:13: FutureWarning:

```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x=list(counts), y=list(words), palette='viridis')
```



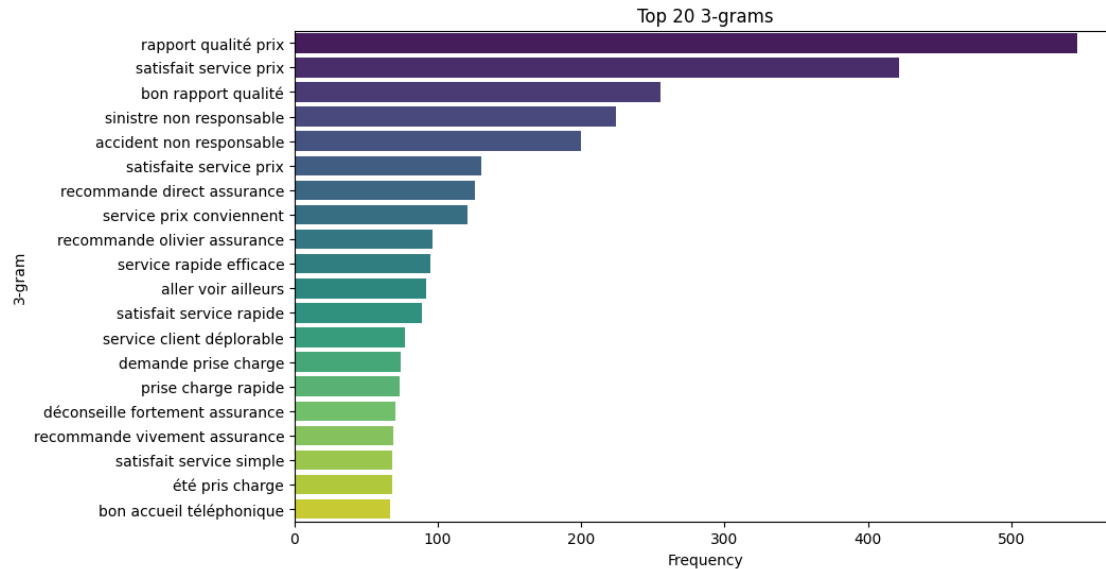
```
[28]: # Same for trigrams instead of bigrams
plot_ngrams(full_data['avis'].dropna(), n=3, top=20)
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/feature_extraction/text.py:402:
UserWarning: Your stop_words may be inconsistent with your preprocessing.
Tokenizing the stop words generated tokens ['neuf', 'qu', 'quelqu'] not in
stop_words.
```

```
warnings.warn(
<ipython-input-27-1f11099f67e0>:13: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x=list(counts), y=list(words), palette='viridis')
```



1.0.3 Handling Missing Data

```
[29]: full_data.isnull().sum()
```

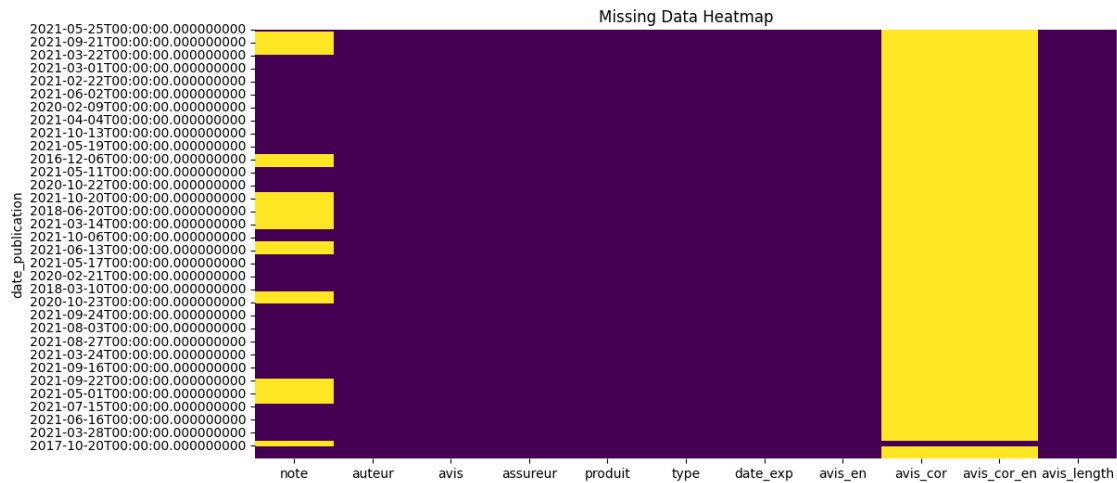
```
[29]: note          10331
auteur            1
avis              0
assureur          0
produit           0
type              0
date_exp          0
avis_en           2
avis_cor          34000
avis_cor_en       34004
avis_length       0
dtype: int64
```

How we handle missing values :

- auteur : the author is not necessary for the rest of the training process so we keep
- avis_en : we will translate the 2 french reviews that are missing in english
- avis_cor + avis_cor_en : we will then correct all french and english reviews to fulfill these columns
- note : We miss a third of the notes for each reviews. We will get rid of missing note reviews for the training of supervised algorithms.

```
[30]: # Visualize missing data
plt.figure(figsize=(12, 6))
sns.heatmap(full_data.isnull(), cbar=False, cmap='viridis')
```

```
plt.title('Missing Data Heatmap')
plt.show()
```



```
[ ]: ## Installer les bibliothèques nécessaires
      # !pip install googletrans==4.0.0rc1
      # !pip install pyspellchecker
```

```
[ ]: import pandas as pd
      import re
      import nltk
      from spellchecker import SpellChecker
      import time
      from multiprocessing import Pool, cpu_count
      from googletrans import Translator

      # Initialize NLTK
      nltk.download('punkt')
      nltk.download('punkt_tab')

      # Initialize spell checkers
      spell_checker_fr = SpellChecker(language='fr')
      spell_checker_en = SpellChecker(language='en')
```

Translation of the 2 missing reviews in english language.

```
[ ]: avis_a_traduire = full_data[full_data['avis_en'].isnull()]
      print(f"Nombre d'avis en français à traduire : {avis_a_traduire.shape[0]}")
```

Nombre d'avis en français à traduire : 2

```
[ ]: translator = Translator()

for index, row in avis_a_traduire.iterrows():
    french_review = row['avis']

    try:
        translated_review = translator.translate(french_review, src='fr',
dest='en').text
    except Exception as e:
        print(f"Erreur lors de la traduction de l'avis à l'index {index} : {e}")
        translated_review = "Erreur de traduction"

    print(f"---\nAvis en Français (index {index}): \n{french_review}\n")
    print(f"Traduction en Anglais: \n{translated_review}\n")

    full_data.at[index, 'avis_en'] = translated_review
```

Avis en Français (index 12999):

La rapidité du traitement de dossier, de l'envoi des documents nécessaires rapide et de l'amabilité des interlocuteurs ainsi que leur connaissance à chaque questions posées. je conseille L'olivier tant bien pour leurs tarifs mais aussi leurs efficacités

Traduction en Anglais:

The speed of file processing, the sending of the necessary necessary documents and the friendliness of the interlocutors as well as their knowledge with each question asked. I recommend the olive tree so much for their prices but also their efficiency

Avis en Français (index 21867):

Aucun

Traduction en Anglais:

None

1.0.4 Cleaning the text and correcting reviews using spellchecker

```
[ ]: # Init correctors
spell_checker_fr = SpellChecker(language='fr')
spell_checker_en = SpellChecker(language='en')

# clean function
def clean_text(text):
    text = str(text)
```

```

text = text.lower()
text = re.sub(r'[^a-zA-Z\s]', '', text, re.I|re.A)
text = text.strip()
return text

def correct_row(row):
    # Correct French text
    words_fr = nltk.word_tokenize(row['avis_clean'], language='french')
    corrected_fr = ' '.join([correction_dict_fr.get(word, word) for word in
↪words_fr])

    # Correct English text
    words_en = nltk.word_tokenize(row['avis_en_clean'], language='english')
    corrected_en = ' '.join([correction_dict_en.get(word, word) for word in
↪words_en])

    return corrected_fr, corrected_en

# Multiprocessing for quicker process
def parallel_correct(data):
    corrected_fr, corrected_en = correct_row(data)
    return corrected_fr, corrected_en

```

```

[ ]: # Clean the text
full_data['avis_clean'] = full_data['avis'].apply(clean_text)
full_data['avis_en_clean'] = full_data['avis_en'].apply(clean_text)

```

Extract every unique word to optimize the correction process as it's way too long !

```

[ ]: # Extract unique words for French and English
unique_words_fr = set()
unique_words_en = set()

for text in full_data['avis_clean'].dropna():
    words = nltk.word_tokenize(text, language='french')
    unique_words_fr.update(words)

for text in full_data['avis_en_clean'].dropna():
    words = nltk.word_tokenize(text, language='english')
    unique_words_en.update(words)

print(f"Unique French words: {len(unique_words_fr)}")
print(f"Unique English words: {len(unique_words_en)}")

```

Unique French words: 43937
Unique English words: 23468

```
[ ]: # Create correction dictionaries
start_time = time.time()
correction_dict_fr = {word: (word if word in spell_checker_fr else
    ↪spell_checker_fr.correction(word) or word) for word in unique_words_fr}
correction_dict_en = {word: (word if word in spell_checker_en else
    ↪spell_checker_en.correction(word) or word) for word in unique_words_en}
end_time = time.time()
print(f"Time to create correction dictionaries: {end_time - start_time:.2f}
    ↪seconds")
```

Time to create correction dictionaries: 19700.46 seconds

```
[ ]: # Prepare data for multiprocessing
data_rows = full_data[['avis_clean', 'avis_en_clean']].to_dict('records')
```

```
[ ]: # Apply corrections in parallel
start_time = time.time()
with Pool(cpu_count()) as pool:
    results = pool.map(parallel_correct, data_rows)
end_time = time.time()
print(f"Time to apply parallel corrections on full data: {end_time - start_time:
    ↪.2f} seconds")
```

Time to apply parallel corrections on full data: 28.15 seconds

```
[ ]: full_data['avis_cor'], full_data['avis_cor_en'] = zip(*results)
```

```
[ ]: full_data.to_csv(os.path.join(project_dir, 'full_data_cleaned_corrected.csv'),
    ↪index=False)
```

Spellchecker doesn't take the context into account. It only corrects the word individually. To take the context into account, we could use more advanced NLP tools like TextBlob or LanguageTool but it would make the task much heavier. That's why we used spellchecker and as you can see, the process still lasted a long...

1.0.5 Creating a dataset containing only rows with not-null "Note" Values :

```
[ ]: print(f"Number of rows before discard : {full_data.shape[0]}")

df = full_data.dropna(subset=['note'])

print(f"Number of rows after discard : {df.shape[0]}")
```

Number of rows before discard : 34435

Number of rows after discard : 24104

```
[ ]: # Saving it :
df.to_csv(os.path.join(project_dir, 'full_data_cleaned_without_note.csv'),
    ↪index=False)
```

2 Supervised Tasks

```
[52]: import os

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, \
    accuracy_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import LinearSVC
from sklearn.pipeline import Pipeline
from sklearn.model_selection import GridSearchCV

from wordcloud import WordCloud
from nltk.corpus import stopwords
import nltk
nltk.download('stopwords')
nltk.download('punkt')
nltk.download('punkt_tab')

from spacy.lang.fr.stop_words import STOP_WORDS as fr_stop
from spacy.lang.en.stop_words import STOP_WORDS as en_stop
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
```

```
[2]: from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
[46]: project_dir = '/content/drive/MyDrive/Colab Notebooks/A5 - ESILV/Machine_
    ↪ Learning for NLP/Project 2'
cleaned_data_path = os.path.join(project_dir, 'full_data.csv')

df = pd.read_csv(cleaned_data_path)
df.head()
```



```
[46]:
```

	note	auteur \		avis	assureur \
0	1.0	maitre-en-colere-114722			
1	5.0	fredo-102557			
2	4.0	louisonne-f-130215			
3	3.0	stephane-p-128761			
4	1.0	montad-137724			

0	Pour tous les maîtres des animaux\n L'assuranc...		Eca Assurances		
1	25 ans chez AMV pour ma moto et mes voitures, ...		AMV		
2	Je suis satisfait de votre prestation et servi...	L'olivier Assurance			
3	SATISFAIT ET FACILE DE SOUSCRIRE UN CONTRAT EN...		AMV		
4	Je croyais qu'allianz était une bonne assuranc...		Allianz		

	produit	type	date_publication	date_exp \
0	animaux	train	25/05/2021	01/05/2021
1	moto	train	13/01/2021	01/01/2021
2	auto	train	30/08/2021	01/08/2021
3	moto	train	20/08/2021	01/08/2021
4	auto	train	18/10/2021	01/10/2021

		avis_en	avis_cor	avis_cor_en
0	For all animal owners\n ECA insurance And avoi...	NaN	NaN	
1	25 years at AMV for my motorcycle and my cars,...	NaN	NaN	
2	I am satisfied with your performance and servi...	NaN	NaN	
3	SATISFIED AND EASY TO SUBSCRIBE TO AN ONLINE C...	NaN	NaN	
4	I thought Allianz was good insurance but I was...	NaN	NaN	

```
[ ]:
```

```
[48]: print(f"Number of rows before discard : {df.shape[0]}")
df = df.dropna(subset=['note'])
print(f"Number of rows after discard : {df.shape[0]}")
```

Number of rows before discard : 34435

Number of rows after discard : 24104

```
[53]: def clean_review(text, stopwords):
    text = str(text)
    text = text.lower()
    text = re.sub(r'[^a-z\s]', ' ', text)
    text = re.sub(r'\s+', ' ', text).strip()
    tokens = nltk.word_tokenize(text, language='french')
    tokens = [word for word in tokens if word not in stopwords]
    cleaned_text = ' '.join(tokens)
    return cleaned_text
```

```
[56]: # for english clean
def clean_review_en(text, stopwords):
    text = str(text)
    text = text.lower()
    text = re.sub(r'^a-z\s', ' ', text)
    text = re.sub(r'\s+', ' ', text).strip()
    tokens = nltk.word_tokenize(text, language='english')
    tokens = [word for word in tokens if word not in stopwords]
    cleaned_text = ' '.join(tokens)
    return cleaned_text

[54]: df['avis_clean'] = df['avis'].apply(lambda x: clean_review(x, fr_stop))
df['avis_en_clean'] = df['avis_en'].apply(lambda x: clean_review(x, en_stop))

[68]: # Concatenating both values for training
df['fr_en_review'] = df['avis_clean'] + " " + df['avis_en_clean']

[69]: df.head()
```

```
[69]:
```

	note	auteur \
0	1.0	maitre-en-colere-114722
1	5.0	fredo-102557
2	4.0	louisonne-f-130215
3	3.0	stephane-p-128761
4	1.0	montad-137724

		avis	assureur \
0	Pour tous les maîtres des animaux\n L'assuranc...		Eca Assurances
1	25 ans chez AMV pour ma moto et mes voitures, ...		AMV
2	Je suis satisfait de votre prestation et servi...	L'olivier Assurance	
3	SATISFAIT ET FACILE DE SOUSCRIRE UN CONTRAT EN...		AMV
4	Je croyais qu'allianz était une bonne assuranc...		Allianz

	produit	type	date_publication	date_exp \
0	animaux	train	25/05/2021	01/05/2021
1	moto	train	13/01/2021	01/01/2021
2	auto	train	30/08/2021	01/08/2021
3	moto	train	20/08/2021	01/08/2021
4	auto	train	18/10/2021	01/10/2021

		avis_en	avis_cor	avis_cor_en \
0	For all animal owners\n ECA insurance And avoi...	NaN	NaN	
1	25 years at AMV for my motorcycle and my cars,...	NaN	NaN	
2	I am satisfied with your performance and servi...	NaN	NaN	
3	SATISFIED AND EASY TO SUBSCRIBE TO AN ONLINE C...	NaN	NaN	
4	I thought Allianz was good insurance but I was...	NaN	NaN	

```

                                avis_clean \
0  animaux l assurance eca vit rembourse chante d...
1  ans amv moto voitures accueil t l phonique r a...
2  satisfait prestation service prix conviennent ...
3  satisfait facile souscrire contrat ligne conta...
4  croyais qu allianz tait bonne assurance tromp ...

                                avis_en_clean \
0  animal owners eca insurance avoided reimburses...
1  years amv motorcycle cars telephone reception ...
2  satisfied performance service prices suit perf...
3  satisfied easy subscribe online contract quick...
4  thought allianz good insurance wrong accident ...

                                fr_en_review
0  animaux l assurance eca vit rembourse chante d...
1  ans amv moto voitures accueil t l phonique r a...
2  satisfait prestation service prix conviennent ...
3  satisfait facile souscrire contrat ligne conta...
4  croyais qu allianz tait bonne assurance tromp ...

```

2.1 Number of Stars prediction

```
[70]: df.dtypes
```

```

[70]: note                float64
      auteur              object
      avis                object
      assureur            object
      produit             object
      type                object
      date_publication    object
      date_exp            object
      avis_en             object
      avis_cor            object
      avis_cor_en         object
      avis_clean          object
      avis_en_clean       object
      fr_en_review        object
      dtype: object

```

```

[60]: # Distribution of star ratings
plt.figure(figsize=(8,6))
sns.countplot(x='note', data=df, palette='viridis')
plt.title('Distribution of Star Ratings')
plt.xlabel('Number of Stars')
plt.ylabel('Count')

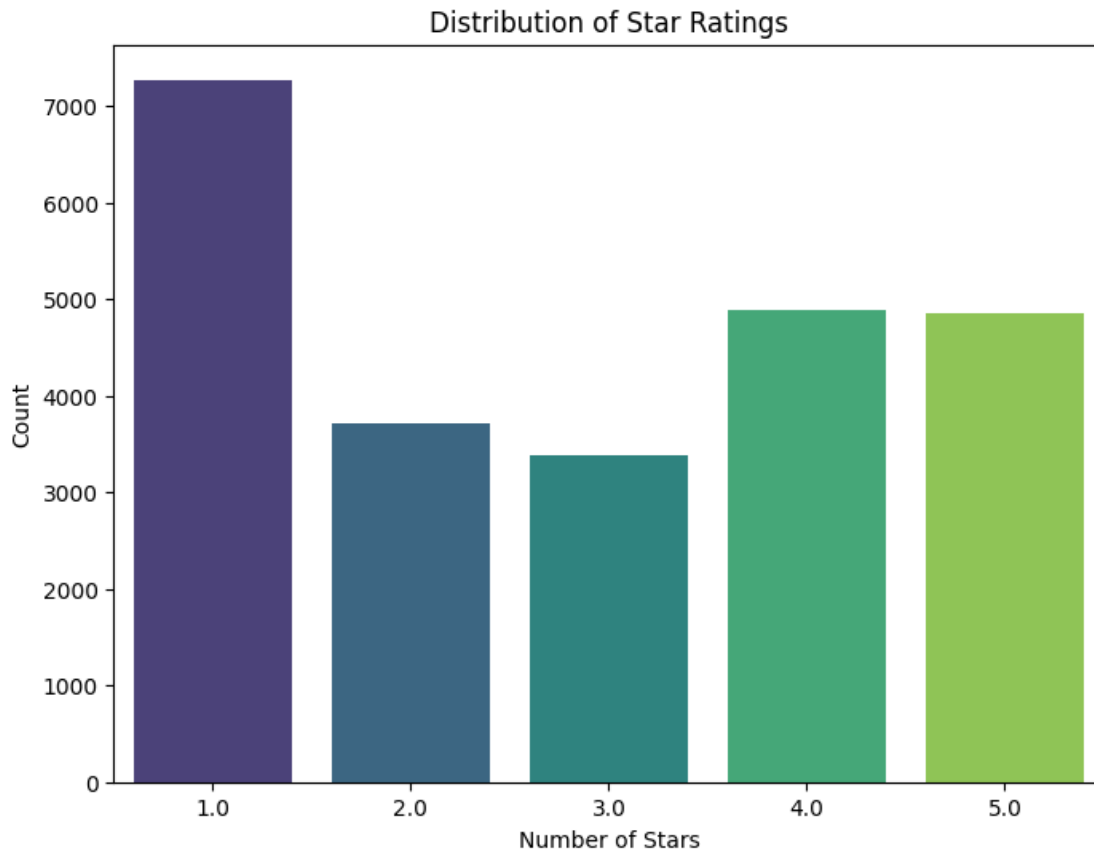
```

```
plt.show()
```

<ipython-input-60-7532f03b72bb>:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(x='note', data=df, palette='viridis')
```



2.1.1 Handling class imbalance

As there are much more notes with value 1 than others, we can balance the number of notes to make sure there is no biases in the prediction.

```
[61]: from sklearn.utils.class_weight import compute_class_weight

# Compute class weights
classes = np.unique(df['note'])
```

```

class_weights = compute_class_weight(class_weight='balanced', classes=classes,
    ↪y=df['note'])
class_weights_dict = {cls: weight for cls, weight in zip(classes,
    ↪class_weights)}
print("Class Weights:", class_weights_dict)

```

```

Class Weights: {1.0: 0.663017466648329, 2.0: 1.2973089343379978, 3.0:
1.4254287403903017, 4.0: 0.9868577277379734, 5.0: 0.993979381443299}

```

```

[71]: # Features and target
X = df['fr_en_review']
y = df['note']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    ↪random_state=42, stratify=y)

```

2.1.2 TF-IDF for “note” : 1-5

Accuracy : 0.49

```

[72]: # Define the pipeline
pipeline_lr = Pipeline([
    ('tfidf', TfidfVectorizer(stop_words=list(fr_stop), max_features=5000)), #
    ↪Adjust max_features as needed
    ('clf', LogisticRegression(class_weight='balanced', random_state=42,
    ↪max_iter=1000))
])

pipeline_lr.fit(X_train, y_train)

```

```

/usr/local/lib/python3.10/dist-packages/sklearn/feature_extraction/text.py:402:
UserWarning: Your stop_words may be inconsistent with your preprocessing.
Tokenizing the stop words generated tokens ['neuf', 'qu', 'quelqu'] not in
stop_words.
    warnings.warn(

```

```

[72]: Pipeline(steps=[('tfidf',
                        TfidfVectorizer(max_features=5000,
                                        stop_words=['nous', 'suivant', 'surtout',
                                                    'moi', 'â', 'tend', 'soi',
                                                    "quelqu'un", 'c', 'par', 'merci',
                                                    'dont', 'ce', 'première', 'votres',
                                                    'elles-mêmes', 'possibles', 'stop',
                                                    'hé', 'des', 'certaine',
                                                    'différente', 'directe', 'qu',
                                                    'parce', 'voici', 'différents',
                                                    'mes', 'deja', 'leur', ...])),
                        ('clf',

```

```
LogisticRegression(class_weight='balanced', max_iter=1000,
                    random_state=42))])
```

```
[73]: y_pred_lr = pipeline_lr.predict(X_test)

# Evaluate the model
print("Logistic Regression Classification Report:")
print(classification_report(y_test, y_pred_lr))

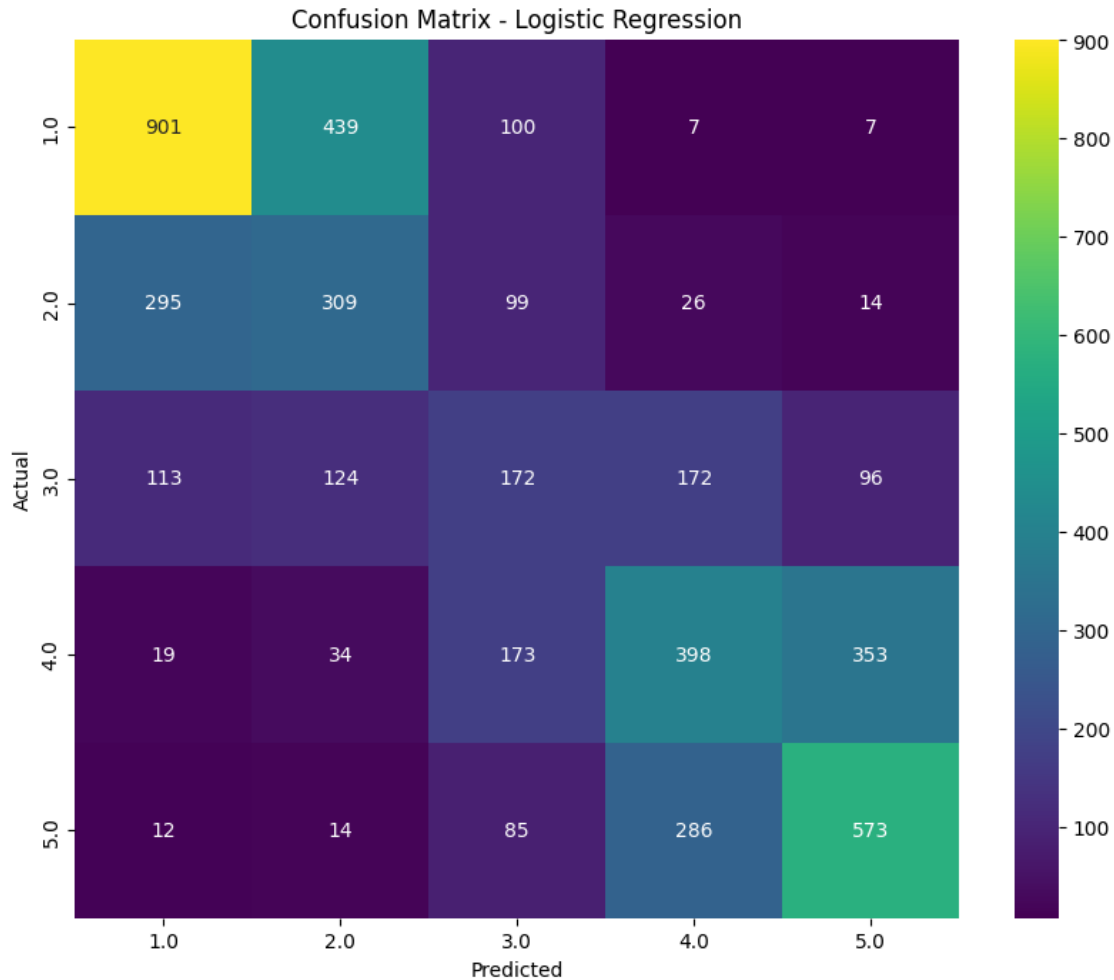
# Confusion Matrix
plt.figure(figsize=(10,8))
sns.heatmap(confusion_matrix(y_test, y_pred_lr), annot=True, fmt='d',
            cmap='viridis',
            xticklabels=classes, yticklabels=classes)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix - Logistic Regression')
plt.show()

# Accuracy Score
accuracy_lr = accuracy_score(y_test, y_pred_lr)
print(f"Logistic Regression Accuracy: {accuracy_lr:.2f}")
```

```
Logistic Regression Classification Report:
              precision    recall  f1-score   support

     1.0         0.67       0.62       0.64       1454
     2.0         0.34       0.42       0.37        743
     3.0         0.27       0.25       0.26        677
     4.0         0.45       0.41       0.43        977
     5.0         0.55       0.59       0.57        970

 accuracy                   0.49       4821
 macro avg         0.46       0.46       0.46       4821
 weighted avg         0.49       0.49       0.49       4821
```



Logistic Regression Accuracy: 0.49

As we seek the right note among 5 possibilities, an accuracy of 49% is quite a good result. If the prediction was perfectly random, we would have 20% of accuracy.

```
[74]: import joblib
      # Saving the model
      model_path = os.path.join(project_dir, 'star_prediction_fr_en.pkl')
      joblib.dump(pipeline_lr, model_path)
```

```
[74]: ['/content/drive/MyDrive/Colab Notebooks/A5 - ESILV/Machine Learning for
      NLP/Project 2/star_prediction_fr_en.pkl']
```

Using this method, we modify the kind of review as we train with a concatenated french and english text. Thus, the performance seems good but while we provide a french review or the exact translation in english, we obtain a different answer errored : * “It’s very bad” is wrongly predicted as a 3 * “C’est très mauvais” is correctly predicted as a 1

Instead, we will train our model on both french and english reviews to make it appropriate for any condition !

```
[75]: df_fr = df[['note', 'avis_clean']].copy()
df_fr = df_fr.rename(columns={'avis_clean': 'review'})

df_en = df[['note', 'avis_en_clean']].copy()
df_en = df_en.rename(columns={'avis_en_clean': 'review'})

df_combined = pd.concat([df_fr, df_en], ignore_index=True)
```

```
[82]: df_combined.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 48208 entries, 0 to 48207
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  -
 0   note    48208 non-null   float64
 1   review  48208 non-null   object
dtypes: float64(1), object(1)
memory usage: 753.4+ KB
```

```
[76]: # Features and target
X = df_combined['review']
y = df_combined['note']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
↳ random_state=42, stratify=y)
```

2.1.3 TF-IDF for “note” : 1-5 for both languages

Accuracy : 0.50

```
[77]: # Define the pipeline
pipeline_lr = Pipeline([
    ('tfidf', TfidfVectorizer(stop_words=list(fr_stop), max_features=5000)), #
↳ Adjust max_features as needed
    ('clf', LogisticRegression(class_weight='balanced', random_state=42,
↳ max_iter=1000))
])

pipeline_lr.fit(X_train, y_train)
```



```
/usr/local/lib/python3.10/dist-packages/sklearn/feature_extraction/text.py:402:
UserWarning: Your stop_words may be inconsistent with your preprocessing.
Tokenizing the stop words generated tokens ['neuf', 'qu', 'quelqu'] not in
stop_words.
```

```
warnings.warn(
```

```
[77]: Pipeline(steps=[('tfidf',
                        TfidfVectorizer(max_features=5000,
                                        stop_words=['nous', 'suivant', 'surtout',
                                                    'moi', 'â', 'tend', 'soi',
                                                    'quelqu'un', 'c', 'par', 'merci',
                                                    'dont', 'ce', 'première', 'votres',
                                                    'elles-mêmes', 'possibles', 'stop',
                                                    'hé', 'des', 'certaine',
                                                    'différente', 'directe', 'qu',
                                                    'parce', 'voici', 'différents',
                                                    'mes', 'déjà', 'leur', ...])),
                      ('clf',
                       LogisticRegression(class_weight='balanced', max_iter=1000,
                                           random_state=42))])
```

```
[78]: y_pred_lr = pipeline_lr.predict(X_test)

# Evaluate the model
print("Logistic Regression Classification Report:")
print(classification_report(y_test, y_pred_lr))

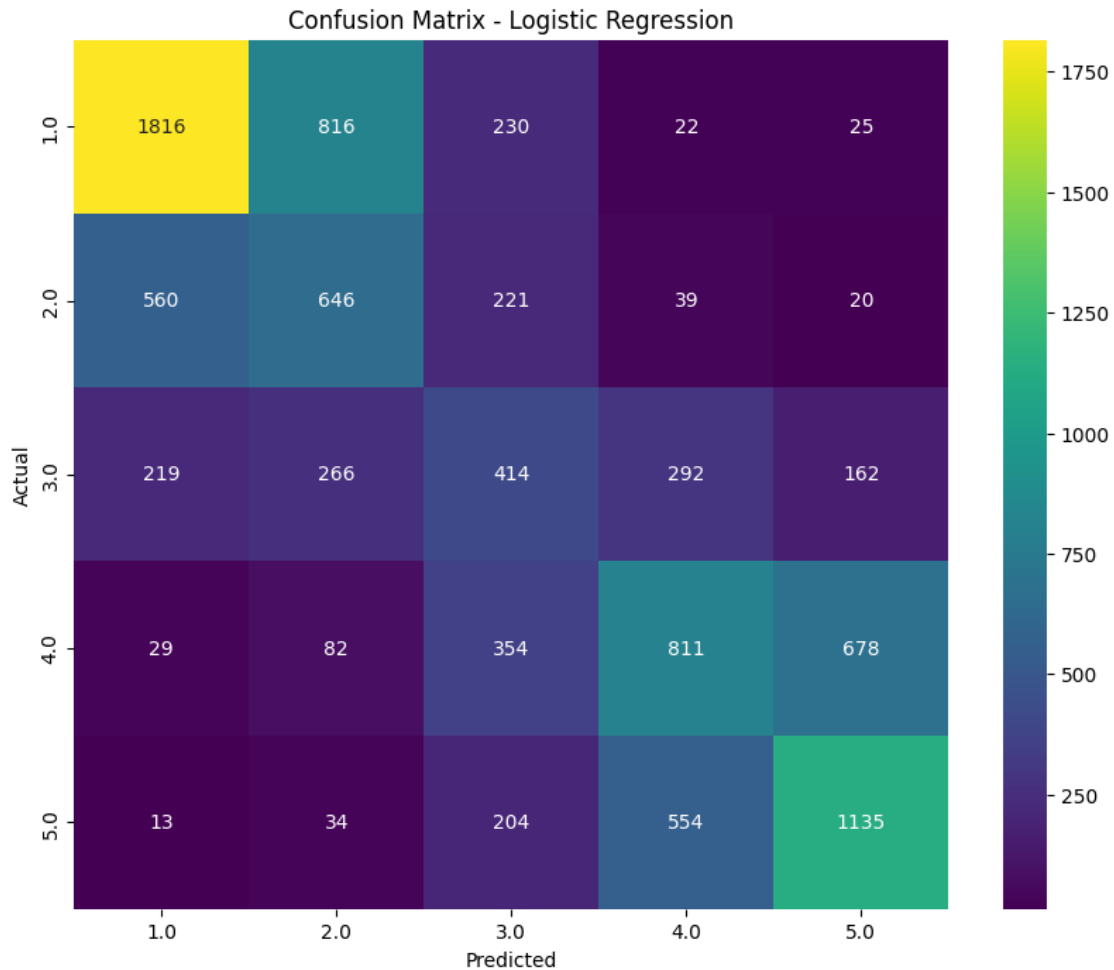
# Confusion Matrix
plt.figure(figsize=(10,8))
sns.heatmap(confusion_matrix(y_test, y_pred_lr), annot=True, fmt='d',
            cmap='viridis',
            xticklabels=classes, yticklabels=classes)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix - Logistic Regression')
plt.show()

# Accuracy Score
accuracy_lr = accuracy_score(y_test, y_pred_lr)
print(f"Logistic Regression Accuracy: {accuracy_lr:.2f}")
```

Logistic Regression Classification Report:

	precision	recall	f1-score	support
1.0	0.69	0.62	0.65	2909
2.0	0.35	0.43	0.39	1486
3.0	0.29	0.31	0.30	1353
4.0	0.47	0.42	0.44	1954

5.0	0.56	0.59	0.57	1940
accuracy			0.50	9642
macro avg	0.47	0.47	0.47	9642
weighted avg	0.51	0.50	0.50	9642



Logistic Regression Accuracy: 0.50

As we seek the right note among 5 possibilities, an accuracy of 49% is quite a good result. If the prediction was perfectly random, we would have 20% of accuracy.

```
[84]: import joblib
      # Saving the model
      model_path = os.path.join(project_dir, 'star_prediction_uni.pkl')
      joblib.dump(pipeline_lr, model_path)
```

```
[84]: ['/content/drive/MyDrive/Colab Notebooks/A5 - ESILV/Machine Learning for  
NLP/Project 2/star_prediction_uni.pkl']
```

2.1.4 TF-IDF for “note” : negative, neutral, positive

Accuracy : 0.75

```
[ ]: # Define the mapping function
def map_to_sentiment(star):
    if star < 3:
        return 'negative'
    elif star == 3:
        return 'neutral'
    else:
        return 'positive'

# Apply the mapping to create a new 'sentiment' column
df['sentiment'] = df['note'].apply(map_to_sentiment)

# Verify the distribution of sentiments
print(df['sentiment'].value_counts())
```

```
sentiment
negative    10987
positive     9735
neutral      3382
Name: count, dtype: int64
```

```
[ ]: # Features and target
X = df['avis']
y = df['sentiment']

# Split into train and test sets (80-20 split), stratify to maintain class
↪distribution
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
[ ]: # Define the pipeline for Logistic Regression
pipeline_lr = Pipeline([
    ('tfidf', TfidfVectorizer(stop_words=list(fr_stop), max_features=5000)),
    ('clf', LogisticRegression(
        class_weight='balanced', # Handle class imbalance
        random_state=42,
        max_iter=1000
    ))
])
```

```
[ ]: pipeline_lr.fit(X_train, y_train)
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/feature_extraction/text.py:402:
UserWarning: Your stop_words may be inconsistent with your preprocessing.
Tokenizing the stop words generated tokens ['neuf', 'qu', 'quelqu'] not in
stop_words.
    warnings.warn(
```

```
[ ]: Pipeline(steps=[('tfidf',
                      TfidfVectorizer(max_features=5000,
                                      stop_words=['seront', 'ouste', 'o', 'chacune',
                                                  'dix-sept', 'd', 'avoir',
                                                  'celles', 'l', 'stop', 'comme',
                                                  'puis', 'fais', 'antérieure',
                                                  'diverses', 'pourrais', 'a', 'me',
                                                  'miens', 'alors', 'suffit',
                                                  'feront', 'un', 'va', 'chez',
                                                  'basee', 'meme', 'tienne',
                                                  'siennes', 'elle', ...])),
                    ('clf',
                     LogisticRegression(class_weight='balanced', max_iter=1000,
                                       random_state=42))])
```

```
[ ]: # Predict on test set
y_pred_lr = pipeline_lr.predict(X_test)

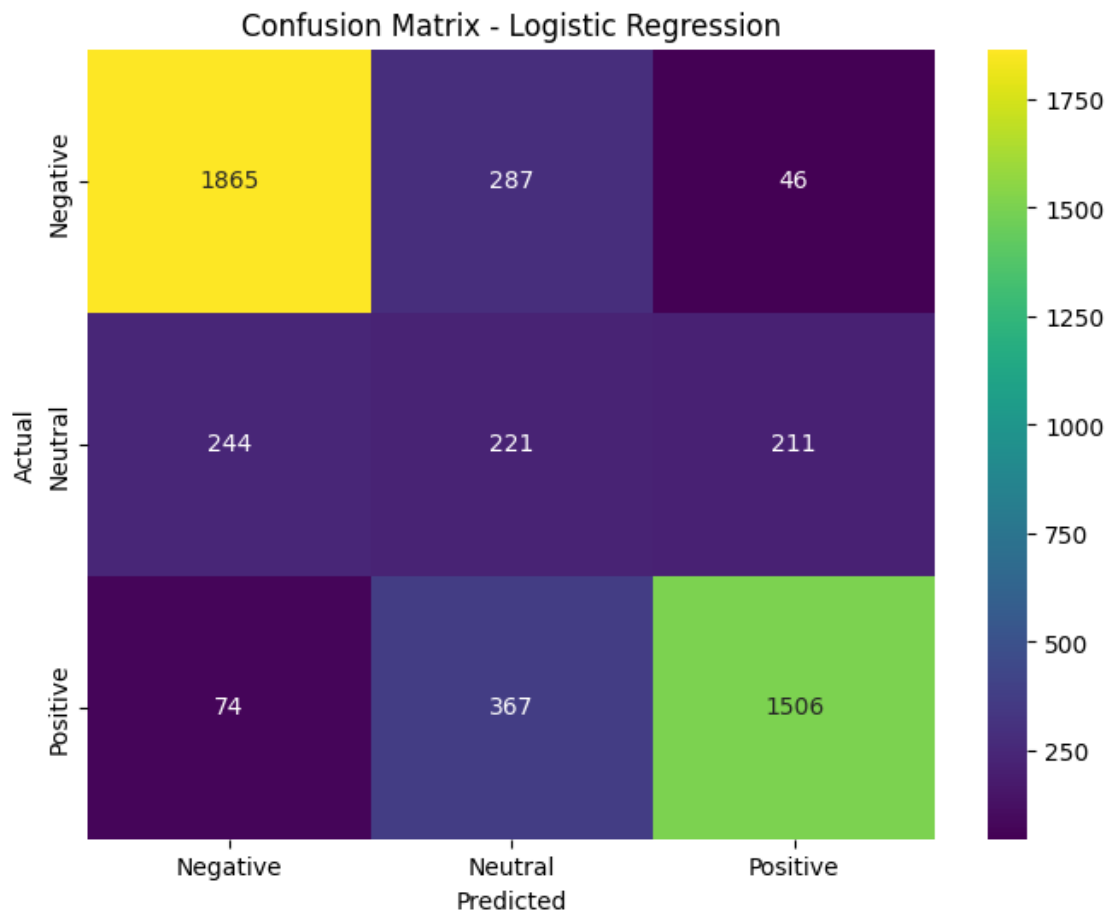
# Evaluate the model
print("Logistic Regression Classification Report:")
print(classification_report(y_test, y_pred_lr))

# Confusion Matrix
plt.figure(figsize=(8,6))
sns.heatmap(confusion_matrix(y_test, y_pred_lr), annot=True, fmt='d',
            cmap='viridis',
            xticklabels=['Negative', 'Neutral', 'Positive'],
            yticklabels=['Negative', 'Neutral', 'Positive'])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix - Logistic Regression')
plt.show()

# Accuracy Score
accuracy_lr = accuracy_score(y_test, y_pred_lr)
print(f"Logistic Regression Accuracy: {accuracy_lr:.2f}")
```

```
Logistic Regression Classification Report:
      precision    recall  f1-score   support
```

negative	0.85	0.85	0.85	2198
neutral	0.25	0.33	0.28	676
positive	0.85	0.77	0.81	1947
accuracy			0.75	4821
macro avg	0.65	0.65	0.65	4821
weighted avg	0.77	0.75	0.76	4821



Logistic Regression Accuracy: 0.75

2.1.5 TF-IDF for “note” : negative OR positive

Accuracy : 0.88

Here, negative concerns notes from 1 to 3 and positive concern notes from 4 to 5.

```
[ ]: # Define the mapping function
def map_to_sentiment(star):
    if star <= 3:
```

```

        return 'negative'
    else:
        return 'positive'

df['binary_sentiment'] = df['note'].apply(map_to_sentiment)
print(df['binary_sentiment'].value_counts())

```

```

binary_sentiment
negative    14369
positive    9735
Name: count, dtype: int64

```

```

[ ]: # Features and target
X = df['avis']
y = df['binary_sentiment']

# Split into train and test sets (80-20 split), stratify to maintain class
↪distribution
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

```

```

[ ]: # Define the pipeline for Logistic Regression
pipeline_lr_binary = Pipeline([
    ('tfidf', TfidfVectorizer(stop_words=list(fr_stop), max_features=5000)),
    ('clf', LogisticRegression(
        class_weight='balanced',
        random_state=42,
        max_iter=1000
    ))
])

```

```

[ ]: pipeline_lr_binary.fit(X_train, y_train)

```

```

/usr/local/lib/python3.10/dist-packages/sklearn/feature_extraction/text.py:402:
UserWarning: Your stop_words may be inconsistent with your preprocessing.
Tokenizing the stop words generated tokens ['neuf', 'qu', 'quelqu'] not in
stop_words.
  warnings.warn(

```

```

[ ]: Pipeline(steps=[('tfidf',
                      TfidfVectorizer(max_features=5000,
                                      stop_words=['seront', 'ouste', 'o', 'chacune',
                                                  'dix-sept', 'd', 'avoir',
                                                  'celles', 'l', 'stop', 'comme',
                                                  'puis', 'fais', 'antérieure',
                                                  'diverses', 'pourrais', 'a', 'me',
                                                  'miens', 'alors', 'suffit',

```

```

        'feront', 'un', 'va', 'chez',
        'basee', 'meme', 'tienne',
        'siennes', 'elle', ...]),
    ('clf',
     LogisticRegression(class_weight='balanced', max_iter=1000,
                        random_state=42))])

```

```

[ ]: # Predict on test set
y_pred_lr_binary = pipeline_lr_binary.predict(X_test)

# Evaluate the model
print("Logistic Regression Classification Report:")
print(classification_report(y_test, y_pred_lr_binary))

# Confusion Matrix
plt.figure(figsize=(8,6))
sns.heatmap(confusion_matrix(y_test, y_pred_lr_binary), annot=True, fmt='d', cmap='viridis',
            xticklabels=['Negative', 'Positive'],
            yticklabels=['Negative', 'Positive'])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix - Logistic Regression')
plt.show()

# Accuracy Score
accuracy_lr = accuracy_score(y_test, y_pred_lr_binary)
print(f"Logistic Regression Accuracy: {accuracy_lr:.2f}")

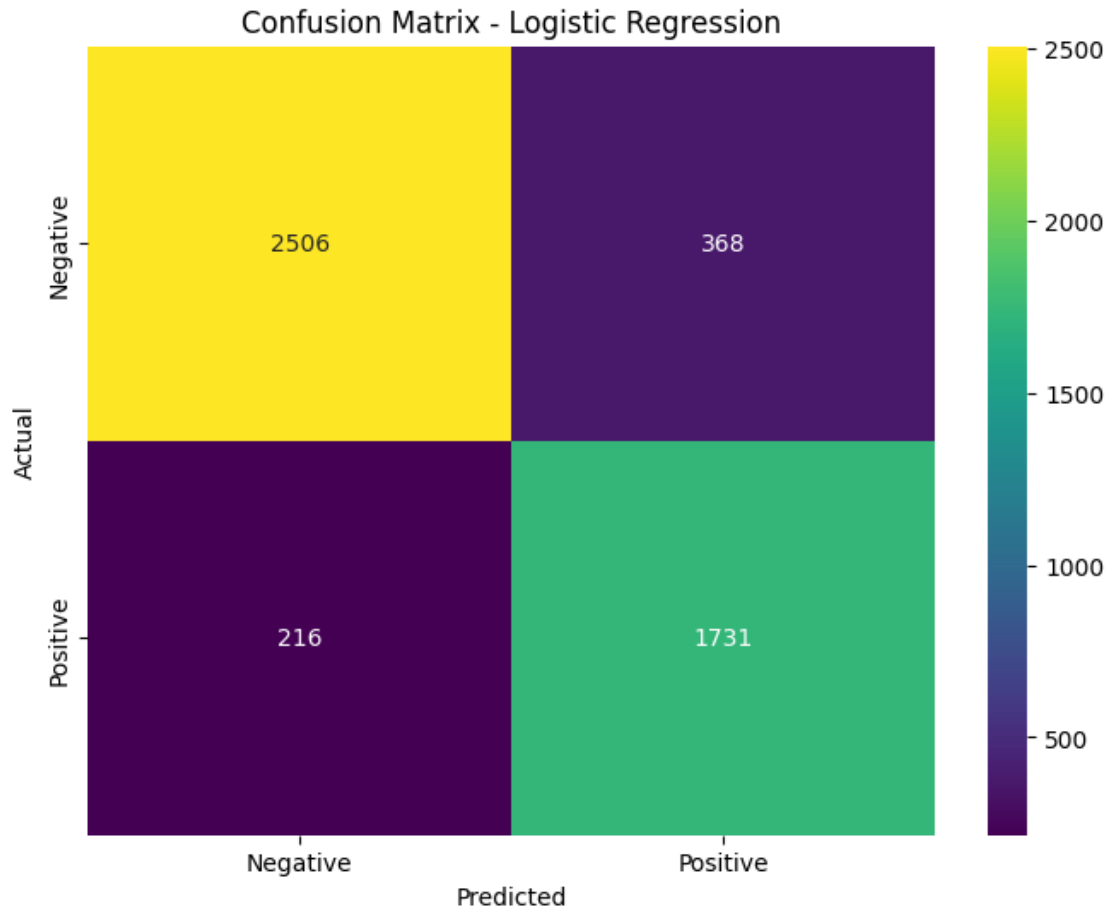
```

```

Logistic Regression Classification Report:

```

	precision	recall	f1-score	support
negative	0.92	0.87	0.90	2874
positive	0.82	0.89	0.86	1947
accuracy			0.88	4821
macro avg	0.87	0.88	0.88	4821
weighted avg	0.88	0.88	0.88	4821



Logistic Regression Accuracy: 0.88

2.1.6 Classical ML - SVM note binary : negative OR positive

Accuracy : 0.87

```
[ ]: # Define the pipeline for SVM
pipeline_svm = Pipeline([
    ('tfidf', TfidfVectorizer(stop_words=list(fr_stop), max_features=5000)),
    ('clf', LinearSVC(class_weight='balanced', random_state=42))
])

# Train the model
pipeline_svm.fit(X_train, y_train)

# Predict on test set
y_pred_svm = pipeline_svm.predict(X_test)

# Evaluate the model
```



```

print("SVM Classification Report:")
print(classification_report(y_test, y_pred_svm))

# Confusion Matrix
plt.figure(figsize=(10,8))
sns.heatmap(confusion_matrix(y_test, y_pred_svm), annot=True, fmt='d',
            cmap='viridis',
            xticklabels=classes, yticklabels=classes)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix - SVM')
plt.show()

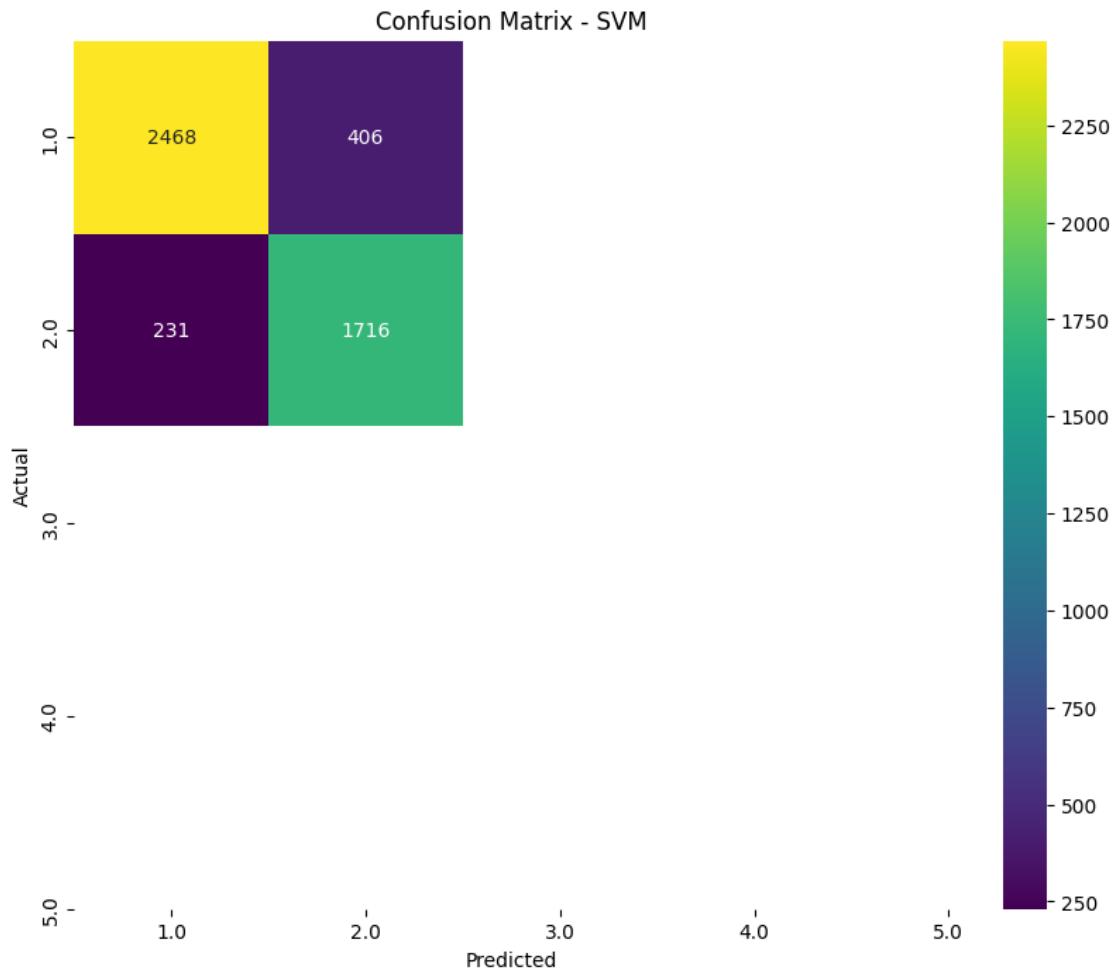
# Accuracy Score
accuracy_svm = accuracy_score(y_test, y_pred_svm)
print(f"SVM Accuracy: {accuracy_svm:.2f}")

```

/usr/local/lib/python3.10/dist-packages/sklearn/feature_extraction/text.py:402:
UserWarning: Your stop_words may be inconsistent with your preprocessing.
Tokenizing the stop words generated tokens ['neuf', 'qu', 'quelqu'] not in
stop_words.
warnings.warn(

SVM Classification Report:

	precision	recall	f1-score	support
negative	0.91	0.86	0.89	2874
positive	0.81	0.88	0.84	1947
accuracy			0.87	4821
macro avg	0.86	0.87	0.86	4821
weighted avg	0.87	0.87	0.87	4821



SVM Accuracy: 0.87

2.2 Model with pre-trained embeddings

```
[ ]: ## Define the path to your cleaned data
## project_dir = '/content/drive/MyDrive/Colab Notebooks/A5 - ESILV/Machine_
↳ Learning for NLP/Project 2'
## cleaned_data_path = os.path.join(project_dir,
↳ 'full_data_cleaned_corrected_parallel.csv')

## Load the cleaned data
## full_data_cleaned = pd.read_csv(cleaned_data_path)

## Display the first few rows
## full_data_cleaned.head()
```

```
[ ]: from transformers import BertTokenizer, BertForSequenceClassification, Trainer, TrainingArguments
import torch
from torch.utils.tensorboard import SummaryWriter
```

```
[ ]: # Define the binary mapping function
def map_to_sentiment_binary(star):
    if star <= 3:
        return 'negative'
    else:
        return 'positive'

# Apply the mapping to create a new 'sentiment' column
df['binary_sentiment'] = df['note'].apply(map_to_sentiment_binary)

# Verify the distribution of sentiments
print(df['binary_sentiment'].value_counts())
```

```
binary_sentiment
negative    14369
positive     9735
Name: count, dtype: int64
```

```
[ ]: # Define the label mapping
label_mapping = {'negative': 0, 'positive': 1}

# Apply the mapping to create numerical labels
df['label'] = df['binary_sentiment'].map(label_mapping)

# Verify the mapping
print(df[['binary_sentiment', 'label']].head())
```

```
          binary_sentiment  label
date_publication
2021-05-25             negative    0
2021-01-13             positive    1
2021-08-30             positive    1
2021-08-20             negative    0
2021-10-18             negative    0
```

```
[ ]: # Initialize the tokenizer
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')

# Initialize the model
model = BertForSequenceClassification.from_pretrained(
    'bert-base-uncased',
    num_labels=2, # Number of classes: negative OR positive
    output_attentions=False,
```

```

        output_hidden_states=False
    )

```

Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-uncased and are newly initialized: ['classifier.bias', 'classifier.weight']
 You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

```

[ ]: # Define the tokenization function
def tokenize_function(examples):
    return tokenizer(
        examples,
        padding='max_length',
        truncation=True,
        max_length=256 # Adjust based on your data
    )

# Tokenize the reviews
tokenized_data = tokenize_function(df['avis'].tolist())

# Extract labels
labels = df['label'].tolist()

```

```

[ ]: !pip install torch

```

```

Requirement already satisfied: torch in /usr/local/lib/python3.10/dist-packages (2.5.1+cu121)
Requirement already satisfied: filelock in /usr/local/lib/python3.10/dist-packages (from torch) (3.16.1)
Requirement already satisfied: typing-extensions>=4.8.0 in /usr/local/lib/python3.10/dist-packages (from torch) (4.12.2)
Requirement already satisfied: networkx in /usr/local/lib/python3.10/dist-packages (from torch) (3.4.2)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.10/dist-packages (from torch) (3.1.5)
Requirement already satisfied: fsspec in /usr/local/lib/python3.10/dist-packages (from torch) (2024.10.0)
Requirement already satisfied: sympy==1.13.1 in /usr/local/lib/python3.10/dist-packages (from torch) (1.13.1)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.10/dist-packages (from sympy==1.13.1->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.10/dist-packages (from jinja2->torch) (3.0.2)

```

```

[ ]: from torch.utils.data import Dataset

class SentimentDataset(Dataset):

```

```

def __init__(self, encodings, labels):
    self.encodings = encodings
    self.labels = labels

def __getitem__(self, idx):
    item = {key: torch.tensor(val[idx]) for key, val in self.encodings.
    ↪items()}
    item['labels'] = torch.tensor(self.labels[idx], dtype=torch.long) # ↪
    ↪Ensure LongTensor
    return item

def __len__(self):
    return len(self.labels)

```

```

[ ]: from torch.utils.data import random_split

# Create the dataset
dataset = SentimentDataset(tokenized_data, labels)

# Define the train-test split
train_size = 0.8
test_size = 0.2

# Split the dataset
train_dataset, test_dataset = random_split(
    dataset,
    [int(train_size * len(dataset)), len(dataset) - int(train_size * ↪
    ↪len(dataset))],
    generator=torch.Generator().manual_seed(42)
)

print(f"Training set size: {len(train_dataset)}")
print(f"Test set size: {len(test_dataset)}")

```

Training set size: 19283

Test set size: 4821

```

[ ]: from transformers import TrainingArguments

training_args = TrainingArguments(
    output_dir=os.path.join(project_dir, 'results'),           # Output directory
    num_train_epochs=3,                                         # Total number of ↪
    ↪training epochs
    per_device_train_batch_size=16,                             # Batch size per ↪
    ↪device during training
    per_device_eval_batch_size=64,                             # Batch size for ↪
    ↪evaluation
)

```

```

    warmup_steps=500,                                # Number of warmup
    ↪steps for learning rate scheduler
    weight_decay=0.01,                                # Strength of
    ↪weight decay
    logging_dir=os.path.join(project_dir, 'logs'),    # Directory for
    ↪storing logs
    logging_steps=10,
    evaluation_strategy='steps',
    eval_steps=50,
    save_steps=100,
    save_total_limit=2,
    load_best_model_at_end=True,                        # Load the best
    ↪model when finished training
    metric_for_best_model='accuracy',                  # Use accuracy to
    ↪evaluate the best model
    greater_is_better=True,
    report_to='tensorboard'                            # Report metrics
    ↪to TensorBoard
)

```

```

/usr/local/lib/python3.10/dist-packages/transformers/training_args.py:1575:
FutureWarning: `evaluation_strategy` is deprecated and will be removed in
version 4.46 of Transformers. Use `eval_strategy` instead
warnings.warn(

```

```

[ ]: from sklearn.metrics import accuracy_score, precision_recall_fscore_support

def compute_metrics_binary(p):
    preds = np.argmax(p.predictions, axis=1)
    labels = p.label_ids
    precision, recall, f1, _ = precision_recall_fscore_support(labels, preds,
    ↪average='binary')
    acc = accuracy_score(labels, preds)
    return {
        'accuracy': acc,
        'f1': f1,
        'precision': precision,
        'recall': recall
    }

```

```

[ ]: from transformers import Trainer

trainer = Trainer(
    model=model,                                # The instantiated
    ↪Transformers model to be trained
    args=training_args,                          # Training arguments,
    ↪defined above
)

```

```

    train_dataset=train_dataset,                # Training dataset
    eval_dataset=test_dataset,                  # Evaluation dataset
    compute_metrics=compute_metrics_binary,     # Function to compute
↪metrics
    tokenizer=tokenizer                          # Tokenizer used
)

```

<ipython-input-98-1775cd6f51b3>:3: FutureWarning: `tokenizer` is deprecated and will be removed in version 5.0.0 for `Trainer.\_\_init\_\_`. Use `processing\_class` instead.

```
trainer = Trainer(
```

```
[ ]: # Start training
trainer.train()
```

<IPython.core.display.HTML object>

```
[ ]: %load_ext tensorboard
%tensorboard --logdir /content/drive/MyDrive/Colab\ Notebooks/A5\ -\ ESILV/
↪Machine\ Learning\ for\ NLP/Project\ 2/tensorboard_logs
```

```
[ ]: from torch.utils.tensorboard import SummaryWriter
from sklearn.manifold import TSNE

# Initialize TensorBoard writer
writer = SummaryWriter(log_dir=os.path.join(project_dir, 'tensorboard_logs'))

# Function to extract embeddings from the [CLS] token
def get_cls_embeddings(dataset, model, tokenizer, device='cpu'):
    model.to(device)
    model.eval()
    embeddings = []
    labels = []
    with torch.no_grad():
        for i in range(len(dataset)):
            item = dataset[i]
            inputs = {key: val.unsqueeze(0).to(device) for key, val in item.
↪items() if key != 'labels'}
            outputs = model.bert(**inputs)
            cls_embedding = outputs.last_hidden_state[:, 0, :].cpu().numpy() #↵
↪[CLS] token
            embeddings.append(cls_embedding[0])
            labels.append(item['labels'].item())
    return np.array(embeddings), np.array(labels)

# Extract embeddings from the test dataset
device = 'cuda' if torch.cuda.is_available() else 'cpu'

```

```

cls_embeddings, cls_labels = get_cls_embeddings(test_dataset, model, tokenizer,
↪device)

# Reduce dimensionality with t-SNE
tsne = TSNE(n_components=3, random_state=42)
embeddings_3d = tsne.fit_transform(cls_embeddings)

# Log embeddings to TensorBoard
writer.add_embedding(
    torch.tensor(embeddings_3d),
    metadata=cls_labels,
    label_img=None,
    global_step=0
)

writer.close()

```

```

[ ]: # Make predictions on the test set
y_pred = trainer.predict(test_dataset).predictions
y_pred_labels = np.argmax(y_pred, axis=1)

# True labels
y_true = labels

# Classification report
print("Classification Report:")
print(classification_report(y_true, y_pred_labels, target_names=['Negative',
↪'Neutral', 'Positive']))

```

```

[ ]: # Compute confusion matrix
cm = confusion_matrix(y_true, y_pred_labels)

# Plot confusion matrix
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='viridis',
            xticklabels=['Negative', 'Neutral', 'Positive'],
            yticklabels=['Negative', 'Neutral', 'Positive'])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix - BERT Sentiment Classification')
plt.show()

```

```

[ ]: # Calculate accuracy
accuracy = accuracy_score(y_true, y_pred_labels)
print(f"Accuracy: {accuracy:.2f}")

```



```
[ ]: # Define the save path
model_save_path = os.path.join(project_dir, 'bert_sentiment_model')

# Save the model and tokenizer
model_for_embedding.save_pretrained(model_save_path)
tokenizer.save_pretrained(model_save_path)

print(f"Model and tokenizer saved to {model_save_path}")

[ ]: from transformers import BertTokenizer, BertForSequenceClassification

# Load the tokenizer and model
loaded_tokenizer = BertTokenizer.from_pretrained(model_save_path)
loaded_model = BertForSequenceClassification.from_pretrained(model_save_path)

# Define a prediction function
def predict_sentiment(text):
    inputs = loaded_tokenizer(
        text,
        return_tensors='pt',
        padding=True,
        truncation=True,
        max_length=256
    )
    with torch.no_grad():
        outputs = loaded_model(**inputs)
    logits = outputs.logits
    predicted_class_id = logits.argmax().item()
    id_to_label = {0: 'negative', 1: 'neutral', 2: 'positive'}
    return id_to_label[predicted_class_id]

# Example prediction
sample_review = ["Je suis très satisfait de votre service. Rapide et efficace!"]
predicted_sentiment = predict_sentiment(sample_review)
print(f"Predicted Sentiment: {predicted_sentiment}")
```

3 TensorBoard

```
[12]: !pip install gensim tensorboard nltk torch

import pandas as pd
import numpy as np
import torch
from torch.utils.tensorboard import SummaryWriter
import gensim.downloader as api
import nltk
import re
```

```

import os

from spacy.lang.fr.stop_words import STOP_WORDS as fr_stop
from spacy.lang.en.stop_words import STOP_WORDS as en_stop

# Download NLTK tokenizer model if not already downloaded
nltk.download('punkt')
nltk.download('punkt_tab')

```

Requirement already satisfied: gensim in /usr/local/lib/python3.10/dist-packages (4.3.3)

Requirement already satisfied: tensorboard in /usr/local/lib/python3.10/dist-packages (2.17.1)

Requirement already satisfied: nltk in /usr/local/lib/python3.10/dist-packages (3.9.1)

Requirement already satisfied: torch in /usr/local/lib/python3.10/dist-packages (2.5.1+cu121)

Requirement already satisfied: numpy<2.0,>=1.18.5 in /usr/local/lib/python3.10/dist-packages (from gensim) (1.26.4)

Requirement already satisfied: scipy<1.14.0,>=1.7.0 in /usr/local/lib/python3.10/dist-packages (from gensim) (1.13.1)

Requirement already satisfied: smart-open>=1.8.1 in /usr/local/lib/python3.10/dist-packages (from gensim) (7.1.0)

Requirement already satisfied: absl-py>=0.4 in /usr/local/lib/python3.10/dist-packages (from tensorboard) (1.4.0)

Requirement already satisfied: grpcio>=1.48.2 in /usr/local/lib/python3.10/dist-packages (from tensorboard) (1.69.0)

Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.10/dist-packages (from tensorboard) (3.7)

Requirement already satisfied: packaging in /usr/local/lib/python3.10/dist-packages (from tensorboard) (24.2)

Requirement already satisfied: protobuf!=4.24.0,>=3.19.6 in /usr/local/lib/python3.10/dist-packages (from tensorboard) (4.25.5)

Requirement already satisfied: setuptools>=41.0.0 in /usr/local/lib/python3.10/dist-packages (from tensorboard) (75.1.0)

Requirement already satisfied: six>1.9 in /usr/local/lib/python3.10/dist-packages (from tensorboard) (1.17.0)

Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.10/dist-packages (from tensorboard) (0.7.2)

Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from tensorboard) (3.1.3)

Requirement already satisfied: click in /usr/local/lib/python3.10/dist-packages (from nltk) (8.1.8)

Requirement already satisfied: joblib in /usr/local/lib/python3.10/dist-packages (from nltk) (1.4.2)

Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.10/dist-packages (from nltk) (2024.11.6)

Requirement already satisfied: tqdm in /usr/local/lib/python3.10/dist-packages

```
(from nltk) (4.67.1)
Requirement already satisfied: filelock in /usr/local/lib/python3.10/dist-
packages (from torch) (3.16.1)
Requirement already satisfied: typing-extensions>=4.8.0 in
/usr/local/lib/python3.10/dist-packages (from torch) (4.12.2)
Requirement already satisfied: networkx in /usr/local/lib/python3.10/dist-
packages (from torch) (3.4.2)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.10/dist-packages
(from torch) (3.1.5)
Requirement already satisfied: fsspec in /usr/local/lib/python3.10/dist-packages
(from torch) (2024.10.0)
Requirement already satisfied: sympy==1.13.1 in /usr/local/lib/python3.10/dist-
packages (from torch) (1.13.1)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/usr/local/lib/python3.10/dist-packages (from sympy==1.13.1->torch) (1.3.0)
Requirement already satisfied: wrapt in /usr/local/lib/python3.10/dist-packages
(from smart-open>=1.8.1->gensim) (1.17.0)
Requirement already satisfied: MarkupSafe>=2.1.1 in
/usr/local/lib/python3.10/dist-packages (from werkzeug>=1.0.1->tensorboard)
(3.0.2)

[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
```

[12]: True

```
[3]: from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
[6]: project_dir = '/content/drive/MyDrive/Colab Notebooks/A5 - ESILV/Machine_
↳ Learning for NLP/Project 2'
cleaned_data_path = os.path.join(project_dir, 'full_data.csv')

df = pd.read_csv(cleaned_data_path)
df.head()
```

```
[6]:      note      auteur \
0    1.0  maitre-en-colere-114722
1    5.0      fred-102557
2    4.0  louis-130215
3    3.0  stephane-p-128761
4    1.0  montad-137724
```

```

                                avis      assureur \
0  Pour tous les maîtres des animaux\n L'assuranc...  Eca Assurances
```

1	25 ans chez AMV pour ma moto et mes voitures, ...	AMV
2	Je suis satisfait de votre prestation et servi...	L'olivier Assurance
3	SATISFAIT ET FACILE DE SOUSCRIRE UN CONTRAT EN...	AMV
4	Je croyais qu'allianz était une bonne assuranc...	Allianz

	produit	type	date_publication	date_exp	\
0	animaux	train	25/05/2021	01/05/2021	
1	moto	train	13/01/2021	01/01/2021	
2	auto	train	30/08/2021	01/08/2021	
3	moto	train	20/08/2021	01/08/2021	
4	auto	train	18/10/2021	01/10/2021	

		avis_en	avis_cor	avis_cor_en
0	For all animal owners\n ECA insurance And avoi...	NaN	NaN	
1	25 years at AMV for my motorcycle and my cars,...	NaN	NaN	
2	I am satisfied with your performance and servi...	NaN	NaN	
3	SATISFIED AND EASY TO SUBSCRIBE TO AN ONLINE C...	NaN	NaN	
4	I thought Allianz was good insurance but I was...	NaN	NaN	

```
[ ]: # print(f"Number of rows before discard : {df.shape[0]}")
# df = df.dropna(subset=['note'])
# print(f"Number of rows after discard : {df.shape[0]}")
```

```
[7]: def clean_review(text, stopwords):
    text = str(text)
    text = text.lower()
    text = re.sub(r'[^a-z\s]', ' ', text)
    text = re.sub(r'\s+', ' ', text).strip()
    tokens = nltk.word_tokenize(text, language='french')
    tokens = [word for word in tokens if word not in stopwords]
    cleaned_text = ' '.join(tokens)
    return cleaned_text
```

```
[14]: df['avis_clean'] = df['avis'].apply(lambda x: clean_review(x, fr_stop))
```

```
[15]: # We get all the words of avis_clean column
vocab = set()
for review in df['avis_clean']:
    tokens = nltk.word_tokenize(review, language='french')
    vocab.update(tokens)
vocab = list(vocab)
print(f"Nombre de mots dans le vocabulaire : {len(vocab)}")
```

Nombre de mots dans le vocabulaire : 29617

3.0.1 Usage of Glove for TensorBoard

```
[21]: embedding_model = api.load("glove-wiki-gigaword-50")
      embedding_dim = embedding_model.vector_size
      print(f"Dimension des embeddings : {embedding_dim}")
```

Dimension des embeddings : 50

```
[22]: embedding_matrix = []
      metadata = []
      for word in vocab:
          if word in embedding_model:
              vector = embedding_model[word]
          else:
              vector = np.zeros(embedding_dim)
          embedding_matrix.append(vector)
          metadata.append(word)

      embedding_matrix = np.array(embedding_matrix)
      print("Forme de la matrice d'embeddings :", embedding_matrix.shape)
```

Forme de la matrice d'embeddings : (29617, 50)

```
[23]: # SummaryWriter for TensorBoard
      project_dir = '/content/drive/MyDrive/Colab Notebooks/A5 - ESILV/Machine_
      ↪ Learning for NLP/Project 2'
      log_dir = os.path.join(project_dir, "runs", "word_embeddings")
      writer = SummaryWriter(log_dir=log_dir)
```

```
[24]: # Log embeddings with metadata
      writer.add_embedding(torch.tensor(embedding_matrix), metadata=metadata,
      ↪ tag="word_embeddings")
      writer.close()
```

```
[25]: %load_ext tensorboard
      %tensorboard --logdir "/content/drive/MyDrive/Colab Notebooks/A5 - ESILV/
      ↪ Machine Learning for NLP/Project 2/runs"
```

The tensorboard extension is already loaded. To reload it, use:

```
%reload_ext tensorboard
```

Reusing TensorBoard on port 6006 (pid 6195), started 0:03:39 ago. (Use '!kill_
↪ 6195' to kill it.)

<IPython.core.display.Javascript object>

4 Unsupervised Tasks

4.1 Identify Categories (Themes) or reviews :

- Pricing
- Coverage
- Enrollment
- Customer Service
- Claims Processing
- Cancellation
- or other categories

4.1.1 Trial with K-Means method

```
[ ]: from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
[ ]: project_dir = '/content/drive/MyDrive/Colab Notebooks/A5 - ESILV/Machine_
↳ Learning for NLP/Project 2'
cleaned_data_path = os.path.join(project_dir, 'full_data_cleaned_without_note.
↳ csv')

df = pd.read_csv(cleaned_data_path)
df.head()
```

```
[ ]:      note      auteur \
0    1.0 maitre-en-colere-114722
1    5.0      fredo-102557
2    4.0    louisonne-f-130215
3    3.0    stephane-p-128761
4    1.0    montad-137724
```

```
                                avis      assureur \
0  Pour tous les maîtres des animaux\n L'assuranc...      Eca Assurances
1  25 ans chez AMV pour ma moto et mes voitures, ...      AMV
2  Je suis satisfait de votre prestation et servi...  L'olivier Assurance
3  SATISFAIT ET FACILE DE SOUSCRIRE UN CONTRAT EN...      AMV
4  Je croyais qu'allianz était une bonne assuranc...      Allianz
```

```
      produit  type date_publication  date_exp \
0  animaux  train    25/05/2021  01/05/2021
1     moto  train    13/01/2021  01/01/2021
2     auto  train    30/08/2021  01/08/2021
3     moto  train    20/08/2021  01/08/2021
4     auto  train    18/10/2021  01/10/2021
```

```
avis_en \
```

```

0 For all masters of animals\n ECA insurance and...
1                               Loading...
2                               Loading...
3                               Loading...
4                               Loading...

                                avis_cor \
0 pour tous les mètres des animaux assurance la ...
1 ans chez ami pour ma moto et mes voitures accu...
2 je suis satisfait de votre prestation et servi...
3 satisfait et facile de souscrire un contrat en...
4 je croyais quallianz tait une bonne assurance ...

                                avis_cor_en \
0 for all masters of animals era insurance and a...
1                               loading
2                               loading
3                               loading
4                               loading

                                avis_clean \
0 pour tous les matres des animaux\n lassurance ...
1 ans chez amv pour ma moto et mes voitures accu...
2 je suis satisfait de votre prestation et servi...
3 satisfait et facile de souscrire un contrat en...
4 je croyais quallianz tait une bonne assurance ...

                                avis_en_clean
0 for all masters of animals\n eca insurance and...
1                               loading
2                               loading
3                               loading
4                               loading

```

```

[ ]: # Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import nltk
import re

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA

nltk.download('punkt')

```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
```

```
[ ]: True
```

```
[ ]: def preprocess_text(text):
    text = text.lower()
    text = re.sub(r'\s+', ' ', text)          # collapse multiple spaces
    text = re.sub(r'[\w\s]', '', text)        # remove punctuation
    return text.strip()

df['processed_text'] = df['avis_en'].apply(preprocess_text)
```

```
[ ]: vectorizer = TfidfVectorizer(stop_words='english', max_features=5000)
tfidf_matrix = vectorizer.fit_transform(df['processed_text'])

print("TF-IDF matrix shape:", tfidf_matrix.shape)
```

```
TF-IDF matrix shape: (24104, 5000)
```

```
[ ]: k = 6
km = KMeans(n_clusters=k, random_state=42, n_init=10)
km.fit(tfidf_matrix)
clusters = km.labels_

# Save the cluster labels in our dataframe
df['cluster'] = clusters

print("Cluster counts:")
print(df['cluster'].value_counts())
```

```
Cluster counts:
```

```
cluster
```

```
4    9637
0    5348
1    4301
2    2357
5    1906
3     555
```

```
Name: count, dtype: int64
```

```
[ ]: def get_top_terms_per_cluster(model, vectorizer, n_terms=10):
    # The cluster centroids are in the TF-IDF feature space.
    order_centroids = model.cluster_centers_.argsort()[:, :-1]
    terms = vectorizer.get_feature_names_out()
    cluster_terms = {}
    for i in range(model.n_clusters):
        top_terms = [terms[ind] for ind in order_centroids[i, :n_terms]]
        cluster_terms[i] = top_terms
```



```

        print(f"Cluster {i}: {' '.join(top_terms)}")
    return cluster_terms

print("\nTop terms per cluster:")
cluster_terms = get_top_terms_per_cluster(km, vectorizer, n_terms=10)

```

Top terms per cluster:

Cluster 0: insurance, years, vehicle, car, year, accident, home, contract, responsible, euros

Cluster 1: satisfied, price, service, insurance, recommend, fast, thank, simple, easy, services

Cluster 2: prices, satisfied, service, insurance, recommend, suit, simple, services, attractive, correct

Cluster 3: loading, émeline, electrical, electronic, electronically, element, elements, elevated, elex, eligible

Cluster 4: mutual, contract, insurance, phone, service, customer, months, time, file, flee

Cluster 5: good, price, insurance, satisfied, service, recommend, thank, value, money, fast

- Cluster 0 : The presence of terms like “accident,” “vehicle,” “car,” “home,” and “contract” suggests that these reviews often mention incidents or situations that require to process claims covering. → **Claims Processing**
- Cluster 1 : Reviews in this cluster emphasize positive feelings with words such as “satisfied,” “fast,” “thank,” “simple,” and “easy.” The overall tone appears to relate with a service. → **Customer Service**
- Cluster 2: Here, the term “prices” is prominent with adjectives like “attractive” and “correct.” This cluster concerns reviews that relate to cost and money. → **Pricing**
- Cluster 3: It features a series of terms related to electronics (“electrical,” “electronic,” “electronically”) and words like “eligible.” This suggests that reviews in this cluster may be discussing aspects of coverage or policy details related to electronics or technical elements of the policy. → **Coverage**
- Cluster 4: Terms like “mutual,” “contract,” and “file” point toward issues around paperwork, policy formation, or the enrollment process. “Phone” and “customer” too. → **Enrollment/Contracts**
- Cluster 5: “good,” “satisfied,” “value,” “money,” “fast” are positive adjectives and reinforces a positive customer experience so we chose to assign it to a new category. → **Overall Positive Experience**

4.1.2 Trial with Zero-Shot Method

```
[ ]: !pip install transformers
```

Requirement already satisfied: transformers in /usr/local/lib/python3.10/dist-packages (4.47.1)

Requirement already satisfied: filelock in /usr/local/lib/python3.10/dist-packages (from transformers) (3.16.1)

Requirement already satisfied: huggingface-hub<1.0,>=0.24.0 in /usr/local/lib/python3.10/dist-packages (from transformers) (0.27.1)

Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.10/dist-packages (from transformers) (1.26.4)

Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from transformers) (24.2)

Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.10/dist-packages (from transformers) (6.0.2)

Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.10/dist-packages (from transformers) (2024.11.6)

Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from transformers) (2.32.3)

Requirement already satisfied: tokenizers<0.22,>=0.21 in /usr/local/lib/python3.10/dist-packages (from transformers) (0.21.0)

Requirement already satisfied: safetensors>=0.4.1 in /usr/local/lib/python3.10/dist-packages (from transformers) (0.5.1)

Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.10/dist-packages (from transformers) (4.67.1)

Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.10/dist-packages (from huggingface-hub<1.0,>=0.24.0->transformers) (2024.10.0)

Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.10/dist-packages (from huggingface-hub<1.0,>=0.24.0->transformers) (4.12.2)

Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (3.4.1)

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (3.10)

Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (2.3.0)

Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (2024.12.14)

```
[ ]: # from google.colab import drive
      # drive.mount('/content/drive')
```

Mounted at /content/drive

```
[ ]: # project_dir = '/content/drive/MyDrive/Colab Notebooks/A5 - ESILV/Machine_
      ↪ Learning for NLP/Project 2'
      # cleaned_data_path = os.path.join(project_dir, 'full_data_cleaned_without_note.
      ↪ csv')

      # df = pd.read_csv(cleaned_data_path)
      # df.head()
```

```
[ ]:      note          auteur \
0   1.0  maitre-en-colere-114722
1   5.0          fredo-102557
2   4.0      louisonne-f-130215
3   3.0      stephane-p-128761
4   1.0      montad-137724
```

```
                                avis          assureur \
0  Pour tous les maîtres des animaux\n L'assuranc...      Eca Assurances
1  25 ans chez AMV pour ma moto et mes voitures, ...      AMV
2  Je suis satisfait de votre prestation et servi...  L'olivier Assurance
3  SATISFAIT ET FACILE DE SOUSCRIRE UN CONTRAT EN...      AMV
4  Je croyais qu'allianz était une bonne assuranc...      Allianz
```

```
      produit   type date_publication   date_exp \
0  animaux  train    25/05/2021  01/05/2021
1    moto  train    13/01/2021  01/01/2021
2    auto  train    30/08/2021  01/08/2021
3    moto  train    20/08/2021  01/08/2021
4    auto  train    18/10/2021  01/10/2021
```

```
                                avis_en \
0  For all masters of animals\n ECA insurance and...
1                                Loading...
2                                Loading...
3                                Loading...
4                                Loading...
```

```
                                avis_cor \
0  pour tous les mètres des animaux assurance la ...
1  ans chez ami pour ma moto et mes voitures accu...
2  je suis satisfait de votre prestation et servi...
3  satisfait et facile de souscrire un contrat en...
4  je croyais quallianz tait une bonne assurance ...
```

```
                                avis_cor_en \
0  for all masters of animals era insurance and a...
1                                loading
2                                loading
3                                loading
4                                loading
```

```
                                avis_clean \
0  pour tous les matres des animaux\n lassurance ...
1  ans chez amv pour ma moto et mes voitures accu...
2  je suis satisfait de votre prestation et servi...
3  satisfait et facile de souscrire un contrat en...
```

```
4 je croyais quallianz tait une bonne assurance ...
```

```
                                avis_en_clean
0 for all masters of animals\n eca insurance and...
1                                loading
2                                loading
3                                loading
4                                loading
```

```
[ ]: # def preprocess_text(text):
#     text = text.lower()
#     text = re.sub(r'\s+', ' ', text)          # collapse multiple spaces
#     text = re.sub(r'[\w\s]', '', text)        # remove punctuation
#     return text.strip()

# df['processed_text'] = df['avis_en'].apply(preprocess_text)
```

```
[ ]: from transformers import pipeline
import time
```

Implementation from Hugging Face

```
[ ]: classifier = pipeline("zero-shot-classification", model="facebook/
↳bart-large-mnli")
```

/usr/local/lib/python3.10/dist-packages/huggingface_hub/utils/_auth.py:94:

UserWarning:

The secret `HF\_TOKEN` does not exist in your Colab secrets.

To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>), set it as secret in your Google Colab and restart your session.

You will be able to reuse this secret in all of your notebooks.

Please note that authentication is recommended but still optional to access public models or datasets.

warnings.warn(

config.json: 0%| | 0.00/1.15k [00:00<?, ?B/s]

model.safetensors: 0%| | 0.00/1.63G [00:00<?, ?B/s]

tokenizer_config.json: 0%| | 0.00/26.0 [00:00<?, ?B/s]

vocab.json: 0%| | 0.00/899k [00:00<?, ?B/s]

merges.txt: 0%| | 0.00/456k [00:00<?, ?B/s]

tokenizer.json: 0%| | 0.00/1.36M [00:00<?, ?B/s]

Device set to use cpu

```
[ ]: candidate_labels = ["Pricing", "Coverage", "Enrollment", "Customer Service",
↳"Claims Processing", "Cancellation"]
```

```
[ ]: def classify_review(review_text, candidate_labels):
    # Run the zero-shot classifier on the review text
    result = classifier(review_text, candidate_labels)
    # The classifier returns a dict with keys: 'sequence', 'labels', and
    ↪ 'scores'
    # We take the highest scoring label and its score.
    top_label = result["labels"][0]
    top_score = result["scores"][0]
    return top_label, top_score
```

```
[ ]: df_subset = df.head(1)
    # df_subset = df

    start_time = time.time()

    # Applying classification review
    df_subset[['predicted_theme', 'theme_score']] = df_subset['avis_en'].apply(
        lambda x: pd.Series(classify_review(x, candidate_labels))
    )

    elapsed_time = time.time() - start_time
    print(f"Processed {len(df_subset)} reviews in {elapsed_time:.2f} seconds.")
```

Processed 1 reviews in 54.78 seconds.

<ipython-input-26-873589c0f331>:7: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.

Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df_subset[['predicted_theme', 'theme_score']] = df_subset['avis_en'].apply(
```

<ipython-input-26-873589c0f331>:7: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.

Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df_subset[['predicted_theme', 'theme_score']] = df_subset['avis_en'].apply(
```

```
[ ]: df_subset.head()
```

```
[ ]:      note      auteur \
0    1.0  maitre-en-colere-114722

      avis      assureur produit \
0  Pour tous les maîtres des animaux\n L'assuranc...  Eca Assurances  animaux
```

```

    type date_publication    date_exp \
0  train      25/05/2021  01/05/2021

                                avis_en \
0  For all masters of animals\n ECA insurance and...

                                avis_cor \
0  pour tous les mètres des animaux assurance la ...

                                avis_cor_en \
0  for all masters of animals era insurance and a...

                                avis_clean \
0  pour tous les matres des animaux\n lassurance ...

                                avis_en_clean \
0  for all masters of animals\n eca insurance and...

                                processed_text    cluster predicted_theme \
0  for all masters of animals eca insurance and a...      0      Enrollment

    theme_score
0      0.375506

```

```

[ ]: print(df_subset[['combined_review', 'predicted_theme', 'theme_score']].head(10))

[ ]: output_path = f"{project_dir}/reviews_with_themes.csv"
df_subset.to_csv(output_path, index=False)
print(f"Saved the dataset with predicted themes to {output_path}.")

```